

DATABASE MANAGEMENT SYSTEMS

PROJECT-BASED LEARNING (PBL) REPORT

Small Database Application Development using Python (Flask and MySQL)

PROJECT TITLE

Design and Implementation of a Database Management System for Robotics Laboratory Equipment, Component Booking and Maintenance Management (ROBO-LAB)

Program	B.Tech CSE – Artificial Intelligence and Machine Learning
Section	AI & ML - Rhinos
Faculty	prof. Sahithi
Date of Submission	10/08/26
Academic Year	2026 – 27

Submitted by

Roll Number	Student Name	Mobile	Email ID
25WU0101077	Mohd Sufiyan Naveed	+91-81259-97774	Sufiyan.mohd_2029@woxsen.edu.in

ACKNOWLEDGEMENT

I express my sincere gratitude to **Woxsen University, School of Technology**, for providing me with the opportunity to undertake this Project-Based Learning (PBL) activity as part of the **Database Management Systems (DBMS)** course.

I thank my course faculty, prof. Sahithi, for the guidance, suggestions, and support throughout the design and development of this project. The guidance helped me understand the practical application of database concepts such as ER modelling, relational schema design, normalization, SQL, constraints, queries, and database application development.

This PBL activity has strengthened my understanding of DBMS concepts and provided valuable experience in applying them to a real-world problem. I also thank the laboratory technical staff for their cooperation in defining realistic workflow requirements that shaped the system design.

UNDERTAKING

I hereby declare that the project entitled “**Design and Implementation of a Database Management System for Robotics Laboratory Equipment, Component Booking and Maintenance Management**”, submitted as part of the Project-Based Learning activity for the **Database Management Systems (DBMS)** course, is my individual work carried out under the guidance of the course faculty.

I undertake that the project report, database design, SQL scripts, application source code, results, and supporting materials submitted by me have been prepared for academic purposes. Wherever information, ideas, tools, datasets, or other resources have been referred to, appropriate acknowledgement has been provided.

I further undertake that I am responsible for the correctness and authenticity of the work submitted and that I have followed the academic and project guidelines prescribed by the University.



Signature

Date : 10/08/26

Place : Woxsen University

Name: Mohd Sufiyan Naveed

Roll Number: 25WU0101077

Program: B.Tech CSE – AI & ML

TABLE OF CONTENTS

Contents	Page No.
Abstract	8
LIST OF TABLES	5
LIST OF FIGURES	6
LIST OF ACRONYMS	7
1. PROBLEM STATEMENT, SOLUTION STRATEGY AND ER DIAGRAM	9
1.1 Problem Statement	9
1.2 Objectives	10
1.3 Scope and Assumptions	10
1.4 Proposed Solution Strategy	11
1.5 Functional Requirements	12
1.6 Entity-Relationship Diagram (ERD)	13
1.7 Entity and Relationship Summary	15
2. LOGICAL DATABASE DESIGN	17
2.1 Relational Schema	17
2.2 Keys and Integrity Constraints	18
2.3 Functional Dependencies	20
2.4 Normalization up to Third Normal Form (3NF)	21
2.5 Data Dictionary	22
2.6 Implementation of Logical Design: DDL and Sample Data	27
3. APPLICATION DEVELOPMENT USING LOGICAL DATABASE DESIGN	30
3.1 Application Overview	30
3.2 Development Environment and Database Connectivity	30
3.3 Application Modules and Database Mapping	31
3.4 CRUD Operations	32
3.5 Business Rules, Validation and Error Handling	33
3.6 Reports and Database Operations	34
3.7 Application Screens	36
3.8 Testing and Results	38
4. CONCLUSION	40
4.1 Limitations and Future Enhancements	41
5. REFERENCES	43
6. APPENDICES	44
APPENDIX A - COMPLETE MYSQL DDL/DML/QUERY SCRIPT	44
APPENDIX B - PYTHON FLASK SOURCE CODE REFERENCE	45
APPENDIX C - ADDITIONAL TEST DATA, SCREENSHOTS AND OUTPUTS	46

LIST OF TABLES

Table No.	Table Title	Page No.
Table 1	Scope and Assumptions	10
Table 2	Proposed Solution Strategy	11
Table 3	Functional Requirements	12
Table 4	Entity and Relationship Summary	15
Table 5	Relational Schema	17
Table 6	Keys and Integrity Constraints	18
Table 7	Normalization Status up to 3NF	21
Table 8	Data Dictionary	22
Table 9	Database Implementation Environment	27
Table 10	Representative Records of the EQUIPMENT Table	29
Table 11	Representative Records of the COMPONENTS Table	29
Table 12	Representative Records of the EQUIPMENT_ALLOCATION Table	29
Table 13	Application Development Environment	30
Table 14	Application Module-to-Database Mapping	31
Table 15	CRUD Operations	32
Table 16	Business Rules, Validation and Error Handling	33
Table 17	Reports and Database Operations	35
Table 18	Test Cases	38
Table 19	Output of the Equipment Availability Report	46
Table 20	Output of the Low Stock Components	46

LIST OF FIGURES

Figure No.	Figure Title	Page No.
Figure 1	Entity-Relationship Diagram of the Proposed System	14
Figure 2	Login Screen	36
Figure 3	Role-Tailored Dashboard	36
Figure 4	Equipment Catalog with Status Filtering	36
Figure 5	Component Inventory and Stock Management	37
Figure 6	Booking Management and Approval Screen	37
Figure 7	Component Allocation – Issue and Return	37
Figure 8	Maintenance and Breakdown Management	37
Figure 9	Reports Playground – SQL Output View	38

LIST OF ACRONYMS

Acronym	Expansion
1NF	First Normal Form
2NF	Second Normal Form
3NF	Third Normal Form
ACID	Atomicity, Consistency, Isolation, Durability
API	Application Programming Interface
AWS	Amazon Web Services
BCNF	Boyce-Codd Normal Form
CRUD	Create, Read, Update, Delete
CSS	Cascading Style Sheets
DBMS	Database Management System
DDL	Data Definition Language
DML	Data Manipulation Language
DOF	Degrees of Freedom
ER	Entity-Relationship
FK	Foreign Key
HTML	Hypertext Markup Language
IDE	Integrated Development Environment
IMU	Inertial Measurement Unit
INR	Indian Rupee
IoT	Internet of Things
JS	JavaScript
PBL	Project-Based Learning
PK	Primary Key
QR	Quick Response
RFID	Radio-Frequency Identification
ROS	Robot Operating System
SBC	Single Board Computer
SMS	Short Message Service
SMTP	Simple Mail Transfer Protocol
SQL	Structured Query Language
URL	Uniform Resource Locator

ABSTRACT

The Robotics Laboratory at Woxsen University shares a large inventory of high-value equipment and consumable electronic components among students, faculty and technicians. Tracking this inventory through paper registers and spreadsheets makes it hard to know what is available, who holds which item for which project, when items are due back, how often equipment fails and when stock runs low. This project aims to design and implement a relational database system, ROBO-LAB, for equipment, component booking and maintenance management. An Entity-Relationship model of nineteen entities is converted into a relational schema verified up to Third Normal Form and implemented in MySQL with primary and foreign keys, CHECK constraints, ENUM domains, triggers for automated stock updates and five analytical views. A Python Flask web application provides role-based access for administrators, lab technicians, faculty and students, covering equipment and component catalogues, project teams, booking approval, component issue and return, breakdown and maintenance tracking, calibration, vendors, procurement and notifications. A reports module runs multi-table JOIN, GROUP BY, subquery and UNION queries to produce availability, low-stock, utilisation and maintenance reports. The completed system shows that a normalised database integrated with a web application gives a transparent and auditable way to manage laboratory resources.

Keywords: *Database Management System, Robotics Laboratory, Inventory Management, MySQL, Python Flask, Normalization*

CHAPTER 1

PROBLEM STATEMENT, SOLUTION STRATEGY AND ER DIAGRAM

1.1 Problem Statement

A college Robotics Laboratory serves as a shared facility where students engage in academic experiments, capstone projects, hackathons, and research activities. The laboratory maintains a diverse inventory that includes individually identifiable high-value equipment such as six-degree-of-freedom robotic arms, 3D printers, autonomous mobile robot platforms, drones, oscilloscopes, and CNC milling machines, alongside large quantities of consumable electronic components including microcontroller development boards, sensors, motors, motor drivers, and prototyping supplies. Historically, the management of these resources has relied on paper-based registers, informal verbal communication between students and laboratory staff, and disconnected spreadsheets maintained by individual technicians. This approach makes it practically impossible to determine, at any given moment, which resources are currently available, which student holds a particular component, which academic project the borrowed item is associated with, and when the item is due for return. Overdue returns go undetected, low-stock conditions are noticed only after demand exceeds availability, and breakdown histories for individual equipment units are either incomplete or entirely absent.

The absence of a structured database system further complicates the management of equipment maintenance. Scheduling preventive maintenance requires knowledge of the calibration history of measurement instruments and the service records of mechanical equipment, both of which are difficult to retrieve from paper records. Vendor information and service costs are similarly untracked. These operational deficiencies reduce the efficiency of the laboratory, increase the risk of equipment loss or damage going unrecorded, and create an inequitable environment in which some students are unaware of available resources while others hoard components for extended periods. The ROBO-LAB system is proposed to replace this ad hoc management approach with a structured, relational database-driven web application that provides transparent, role-appropriate access to all resource information for administrators, laboratory technicians, faculty guides, and students.

1.2 Objectives

- **Design** a normalized relational database schema in MySQL to manage robotics laboratory equipment, electronic components, projects, users, bookings, allocations, and maintenance records.
- **Implement** a complete inventory management system tracking equipment availability, component stock levels, minimum stock thresholds, and storage locations.
- **Track** the complete lifecycle of each resource from initial inventory entry through booking, approval, allocation to a student and project, usage, return, breakdown reporting, maintenance, and return to available status.
- **Enforce** database integrity constraints and business rules including stock quantity validation, booking status management, breakdown-triggered equipment status changes, and referential integrity across all foreign key relationships.
- **Develop** a Python Flask web application with role-based access control that supports all CRUD operations and generates analytical reports using JOIN, GROUP BY, subquery, and UNION SQL operations.

1.3 Scope and Assumptions

Table 1 defines the scope of the ROBO-LAB system and lists the assumptions made during its design.

Table 1. Scope and Assumptions

Item	Description
In Scope	• Equipment catalog management (CRUD, status tracking) • Component inventory management (quantity tracking, minimum stock alerts) • User account and role management (Admin, Lab Technician, Faculty, Student) • Project registration and project-member association • Equipment booking with time-slot management and faculty approval workflow • Component allocation (issue) to students and projects • Component return recording with condition assessment • Usage logging for equipment sessions • Breakdown reporting and severity classification • Maintenance job creation, assignment to vendors, and completion recording • Calibration schedule management and due-date alerts • Vendor management with service history • Notification management for booking approvals and stock alerts • Procurement requisition and reorder management • Report generation: availability, allocations, low stock, maintenance, utilization, project costs, calibration • Database triggers for automated stock deduction and status updates • SQL views for frequently executed analytical queries

Item	Description
Out of Scope / Exclusions	• Physical IoT sensor integration or automated hardware status monitoring • Real-time RFID or barcode-based equipment tracking • Online payment processing or e-commerce purchasing • Automated email or SMS notification delivery • Native mobile application development • Cloud deployment or multi-institution access • Advanced role-based permission granularity beyond four defined roles
Assumptions	• Laboratory staff are responsible for maintaining accurate inventory records. • All users have valid institutional accounts created by an administrator. • Equipment and component records are entered by authorized staff only (Admin or Lab Technician role). • The MySQL database instance (v8.x) is accessible to the Flask application server. • This system is an academic prototype and is not designed for high-concurrency production load. • SQLite is used as an automatic fallback database for local development and testing when MySQL is unavailable.

1.4 Proposed Solution Strategy

Table 2 summarises how the proposed system addresses the problem, covering its users, modules, workflow, business rules and expected outputs.

Table 2. Proposed Solution Strategy

Solution Element	Project-Specific Description
Primary Users / Roles	Admin – Full system management including users, equipment, components, laboratories, vendors, and reports. Lab Technician – Equipment issuance, returns, status updates, breakdown recording, and maintenance management. Faculty – Booking approval or rejection, project guidance, and report viewing. Student – Equipment search, booking requests, component allocation viewing, and breakdown reporting.
Major Application Modules	Authentication & Registration; Dashboard (role-tailored); Equipment Catalog; Component Inventory; Projects & Teams; Bookings & Approvals; Allocations (Issue/Return); Usage Logs; Maintenance & Breakdowns; Calibrations; Vendors; Notifications; Procurement/Requisitions; Reports.
Proposed Workflow	A student logs into the application and searches for available equipment. The Flask application queries the EQUIPMENT and EQUIPMENT_CATEGORIES tables via a parameterized SELECT-JOIN statement. The student selects an available item and submits a booking request linked to a project; the booking is stored in BOOKINGS with status PENDING. An authorized faculty member reviews the request and approves or rejects it, updating the booking status to APPROVED or REJECTED. Upon approval, the equipment session begins (status IN_USE), automatically updating the EQUIPMENT status field and creating a USAGE_LOGS entry. For electronic components, a lab technician issues the required quantity directly through the EQUIPMENT_ALLOCATION table; a database trigger automatically deducts the issued quantity from COMPONENTS.available_quantity. When the student returns components, a RETURNS record is inserted; the trigger restores available stock conditionally based on the returned condition. Damaged or faulty equipment generates a BREAKDOWNS entry; equipment with High or Critical severity is automatically marked as Maintenance-status, blocking future bookings. A maintenance job is created in MAINTENANCE_JOBS, optionally linked to a vendor in VENDORS. Upon maintenance completion, the equipment status reverts to Available. All events contribute to an auditable timeline per equipment unit.

Solution Element	Project-Specific Description
Important Business Rules	Only Available-status equipment may be booked. Component available quantity must not become negative. Returned quantity must not exceed allocated quantity. Equipment under Maintenance cannot be allocated. Completion date of a maintenance job must not precede the start date. Unique serial numbers must be enforced across equipment. Foreign key constraints prevent orphaned records. Parameterized SQL queries prevent injection.
Expected Reports / Outputs	Equipment Availability Report; Active Component Allocations; Low Stock Inventory Alert; Maintenance History with Vendor Costs; Equipment Utilization (Booking Frequency); Project Component Cost Analysis; Breakdown Frequency by Equipment; Calibration Due-Date Audit.

1.5 Functional Requirements

The functional requirements of the system are listed in Table 3.

Table 3. Functional Requirements

Req. ID	Functional Requirement	Remarks / Priority
FR-01	The system shall support user registration, login, and role-based access control for Admin, Lab Technician, Faculty, and Student roles.	High
FR-02	The system shall allow authorized users to add, view, update, and manage equipment records including equipment ID, name, category, laboratory, serial number, purchase date, warranty, status, and location.	High
FR-03	The system shall manage component inventory including component ID, name, category, total quantity, available quantity, minimum stock threshold, unit cost, and storage bin.	High
FR-04	The system shall support keyword-based search and status/category filtering for both equipment and components.	High
FR-05	The system shall support project registration, associating a guide faculty member and allowing the addition of student team members.	High
FR-06	Students and authorized staff shall be able to submit equipment booking requests specifying equipment, date, time slot, project, and purpose.	High
FR-07	Faculty and Admin roles shall be able to approve or reject pending booking requests, with the approver identity recorded.	High
FR-08	Lab Technicians and Admin roles shall be able to issue electronic components to students against a specific project, with quantity, issue date, and expected return date recorded.	High
FR-09	The system shall record equipment usage sessions (start and end timestamps, associated project) when a booking transitions to IN USE status.	Medium
FR-10	The system shall record component returns, including returned quantity, condition status (Good, Damaged, Burnt/Faulty, Missing Parts), verifying staff, and notes. Stock is restored automatically for Good and Damaged conditions.	High
FR-11	Any user shall be able to report a breakdown for equipment, specifying the problem description and severity. High or Critical severity shall automatically set the equipment status to Maintenance.	High
FR-12	Lab Technicians and Admin shall be able to create maintenance jobs linked to breakdowns and vendors, and record completion with action taken and cost.	High
FR-13	The system shall maintain a vendor registry with contact details and service type, and display accumulated service cost per vendor.	Medium
FR-14	The system shall manage calibration records for equipment, tracking last calibration date, next due date, certifying authority, and status (Valid, Due Soon, Overdue).	Medium
FR-15	The system shall generate and display in-application notifications for booking approvals, stock alerts, and maintenance events.	Medium

Req. ID	Functional Requirement	Remarks / Priority
FR-16	The system shall execute and display pre-defined SQL reports covering equipment availability, component allocations, low stock, maintenance history, booking utilization, project component costs, breakdown frequency, and calibration schedules.	High
FR-17	The system shall support procurement requisition management including priority classification, multi-vendor comparison links, quantity tracking, and status lifecycle management.	Medium

1.6 Entity-Relationship Diagram (ERD)

Figure 1 presents the final conceptual design of the ROBO-LAB system as an ER diagram. All entities shown correspond directly to tables in the implemented MySQL schema. Relationships are labeled with their cardinalities. The diagram is referenced throughout Chapter 2 and must be read alongside the relational schema in Table 5.



Figure 1. Entity-Relationship Diagram of the Proposed System

ER Diagram Design Notes:

The design distinguishes two fundamentally different resource types. EQUIPMENT represents individually identifiable high-value assets (each tracked by a unique equipment_id and serial_number). COMPONENTS represents quantity-tracked items that exist in multiple units and are issued in specified quantities to students. BOOKINGS provides time-slot reservation for EQUIPMENT. EQUIPMENT_ALLOCATION records the issuance of specific COMPONENT quantities to students against PROJECTS. RETURNS references EQUIPMENT_ALLOCATION to record actual return events. BREAKDOWNS and MAINTENANCE_JOBS form a linked repair-lifecycle chain connected to EQUIPMENT. CALIBRATIONS tracks metrology schedules for measurement equipment. VENDORS supports the MAINTENANCE_JOBS relationship. NOTIFICATIONS delivers system events to individual USERS. REQUISITIONS manages procurement requests for low-stock COMPONENTS.

1.7 Entity and Relationship Summary

Table 4 summarises the entities of the ER model, their major attributes and the cardinality of the key relationships.

Table 4. Entity and Relationship Summary

Entity / Relationship	Major Attributes / Cardinality	Purpose / Rule
ROLES	PK: role_id; role_name (UNIQUE, NOT NULL)	Defines the four system roles: Admin, Lab Technician, Faculty, Student.
USERS	PK: user_id; FK: role_id; email (UNIQUE); roll_number (UNIQUE)	Stores all system users with role assignment. email and roll_number are unique identifiers.
LABORATORIES	PK: lab_id; FK: incharge_id → USERS; lab_name (UNIQUE)	Represents physical laboratory spaces. Each lab has a designated incharge from USERS.
EQUIPMENT_CATEGORIES	PK: category_id; category_name (UNIQUE)	Classifies equipment into types such as Robotic Arms, 3D Printers, Autonomous Robots.
EQUIPMENT	PK: equipment_id (VARCHAR); FK: category_id, laboratory_id; serial_number (UNIQUE); status ENUM	Individually identifiable equipment assets. Status controls booking and allocation eligibility.
COMPONENT_CATEGORIES	PK: category_id; category_name (UNIQUE)	Classifies components into types such as Microcontrollers, Sensors, Motors.
COMPONENTS	PK: component_id (VARCHAR); FK: category_id; CHECK available_quantity ≥ 0; CHECK total_quantity ≥ 0	Quantity-tracked components. available_quantity drives issuance eligibility.
PROJECTS	PK: project_id; FK: guide_faculty_id → USERS; status ENUM	Represents student academic projects. A faculty member guides each project.
PROJECT_MEMBERS	PK: (project_id, user_id); FK: project_id, user_id	Associative entity linking USERS to PROJECTS. A student may be a member of multiple projects; a project has multiple members. M:N resolved.
BOOKINGS	PK: booking_id; FK: equipment_id, user_id, project_id, approved_by → USERS; status ENUM	Records time-slot reservations for equipment. Status lifecycle: PENDING → APPROVED/REJECTED → IN USE → RETURNED.
EQUIPMENT_ALLOCATION	PK: allocation_id; FK: component_id, user_id, project_id; CHECK quantity > 0	Records issuance of a specified quantity of a component to a student for a project. Triggers decrement available_quantity.

Entity / Relationship	Major Attributes / Cardinality	Purpose / Rule
USAGE_LOGS	PK: log_id; FK: equipment_id, user_id, project_id; start_timestamp, end_timestamp	Captures detailed time-based usage sessions for equipment, linked to bookings.
RETURNS	PK: return_id; FK: allocation_id, verified_by → USERS; condition_status ENUM	Records the physical return of components. Trigger conditionally restores available_quantity.
VENDORS	PK: vendor_id; phone NOT NULL	Stores service vendor contact and specialization for maintenance.
BREAKDOWNS	PK: breakdown_id; FK: equipment_id, reported_by → USERS; severity ENUM; status ENUM	Records equipment failures. High/Critical severity triggers EQUIPMENT status change to Maintenance.
MAINTENANCE_JOBS	PK: job_id; FK: equipment_id, breakdown_id → BREAKDOWNS, vendor_id → VENDORS; CHECK cost ≥ 0	Tracks repair activities. Links to a breakdown cause and a servicing vendor. Status lifecycle: Scheduled → In Progress → Completed.
CALIBRATIONS	PK: calibration_id; FK: equipment_id; status ENUM (Valid, Due Soon, Overdue)	Tracks calibration schedules for measurement instruments. next_due_date drives due-date alerts.
NOTIFICATIONS	PK: notification_id; FK: user_id; notification_type ENUM; is_read BOOLEAN	Stores system-generated events for individual users. Unread count is displayed in the navigation bar.
REQUISITIONS	PK: requisition_id; FK: component_id → COMPONENTS (nullable), requested_by → USERS; priority ENUM; status ENUM	Manages procurement requests for low-stock or new components with multi-vendor comparison links.
USERS ↔ ROLES	M:1 — Many users may have the same role	FK: USERS.role_id → ROLES.role_id
EQUIPMENT ↔ EQUIPMENT_CATEGORIES	M:1 — Many equipment items belong to one category	FK: EQUIPMENT.category_id → EQUIPMENT_CATEGORIES.category_id
COMPONENTS ↔ COMPONENT_CATEGORIES	M:1 — Many components belong to one category	FK: COMPONENTS.category_id → COMPONENT_CATEGORIES.category_id
BOOKINGS ↔ EQUIPMENT	M:1 — Many bookings for one equipment item	FK: BOOKINGS.equipment_id → EQUIPMENT.equipment_id
EQUIPMENT_ALLOCATION ↔ COMPONENTS	M:1 — Many allocations for one component type	FK: EQUIPMENT_ALLOCATION.component_id → COMPONENTS.component_id
RETURNS ↔ EQUIPMENT_ALLOCATION	1:1 (per return event) — A return references one allocation	FK: RETURNS.allocation_id → EQUIPMENT_ALLOCATION.allocation_id
MAINTENANCE_JOBS ↔ BREAKDOWNS	1:1 (optional) — A maintenance job may resolve one breakdown	FK: MAINTENANCE_JOBS.breakdown_id → BREAKDOWNS.breakdown_id

CHAPTER 2

LOGICAL DATABASE DESIGN

2.1 Relational Schema

The 19 relations derived from the ER model are listed in Table 5, with their primary and foreign keys.

Table 5. Relational Schema

S.No	Table Name	Purpose	Primary Key	Foreign Keys
1	ROLES	Defines user role types	role_id	—
2	USERS	Stores all system user accounts	user_id	role_id → ROLES
3	LABORATORIES	Represents physical lab rooms	lab_id	incharge_id → USERS
4	EQUIPMENT_CATEGORIES	Classifies equipment types	category_id	—
5	EQUIPMENT	Individual equipment assets	equipment_id (VARCHAR)	category_id → EQUIPMENT_CATEGORIES; laboratory_id → LABORATORIES
6	COMPONENT_CATEGORIES	Classifies component types	category_id	—
7	COMPONENTS	Quantity-tracked electronic components	component_id (VARCHAR)	category_id → COMPONENT_CATEGORIES
8	PROJECTS	Academic / research projects	project_id	guide_faculty_id → USERS
9	PROJECT_MEMBERS	M:N association of USERS to PROJECTS	(project_id, user_id) – composite	project_id → PROJECTS; user_id → USERS
10	BOOKINGS	Time-slot equipment reservations	booking_id	equipment_id → EQUIPMENT; user_id → USERS; project_id → PROJECTS; approved_by → USERS
11	EQUIPMENT_ALLOCATION	Component issuance to students/projects	allocation_id	component_id → COMPONENTS; user_id → USERS; project_id → PROJECTS
12	USAGE_LOGS	Equipment session time logs	log_id	equipment_id → EQUIPMENT; user_id → USERS; project_id → PROJECTS
13	RETURNS	Records of component returns	return_id	allocation_id → EQUIPMENT_ALLOCATION; verified_by → USERS
14	VENDORS	External service vendors	vendor_id	—
15	BREAKDOWNS	Equipment failure reports	breakdown_id	equipment_id → EQUIPMENT; reported_by → USERS
16	MAINTENANCE_JOBS	Repair and maintenance activities	job_id	equipment_id → EQUIPMENT; breakdown_id → BREAKDOWNS; vendor_id → VENDORS
17	CALIBRATIONS	Calibration schedule records	calibration_id	equipment_id → EQUIPMENT
18	NOTIFICATIONS	User-targeted system messages	notification_id	user_id → USERS
19	REQUISITIONS	Component procurement requests	requisition_id	component_id → COMPONENTS (nullable); requested_by → USERS

2.2 Keys and Integrity Constraints

Table 6 lists the key and integrity constraints defined in the schema and the business rule each one enforces.

Table 6. Keys and Integrity Constraints

Table Name	Key / Constraint	Attribute(s)	Business Rule Enforced
All 19 tables	PRIMARY KEY	See Table 5	Guarantees unique identification of every record in every table.
USERS	FOREIGN KEY	role_id → ROLES	A user must be assigned a valid defined role; ON UPDATE CASCADE propagates role_id changes.
LABORATORIES	FOREIGN KEY	incharge_id → USERS	The laboratory incharge must exist in USERS; ON DELETE SET NULL allows lab to exist without incharge.
EQUIPMENT	FOREIGN KEY	category_id → EQUIPMENT_CATEGORIES	Every equipment record must belong to a valid category.
EQUIPMENT	FOREIGN KEY	laboratory_id → LABORATORIES	Every equipment record must be assigned to a valid laboratory.
BOOKINGS	FOREIGN KEY	equipment_id → EQUIPMENT	A booking must reference existing equipment; orphaned bookings are prevented.
BOOKINGS	FOREIGN KEY	approved_by → USERS	The approver must be a valid user; ON DELETE SET NULL allows booking to remain if approver is deleted.
EQUIPMENT_ALLOCATION	FOREIGN KEY	component_id → COMPONENTS	An allocation must reference an existing component.
EQUIPMENT_ALLOCATION	FOREIGN KEY	project_id → PROJECTS	Component allocation must be linked to an existing project.
RETURNS	FOREIGN KEY	allocation_id → EQUIPMENT_ALLOCATION	A return record must reference a valid allocation; ON DELETE CASCADE removes return if allocation is deleted.
MAINTENANCE_JOBS	FOREIGN KEY	breakdown_id → BREAKDOWNS	Maintenance may optionally link to a breakdown; ON DELETE SET NULL allows job to exist without breakdown.
MAINTENANCE_JOBS	FOREIGN KEY	vendor_id → VENDORS	Maintenance may optionally link to a vendor; ON DELETE SET NULL retains job history if vendor is deleted.
NOTIFICATIONS	FOREIGN KEY	user_id → USERS	Notifications are user-specific; ON DELETE CASCADE removes notifications when the user is deleted.
PROJECT_MEMBERS	FOREIGN KEY	project_id, user_id	ON DELETE CASCADE removes membership records if the project or user is deleted.
USERS	UNIQUE	email	No two users may share an email address.
USERS	UNIQUE	roll_number	Roll/staff numbers are unique identifiers; NULL permitted for users without institutional IDs.
ROLES	UNIQUE	role_name	Each role name must be distinct.
LABORATORIES	UNIQUE	lab_name	Each laboratory must have a unique name.
EQUIPMENT	UNIQUE	serial_number	No two equipment assets may share a serial number (enforces asset identity).
EQUIPMENT_CATEGORIES	UNIQUE	category_name	Category names must be distinct.
COMPONENT_CATEGORIES	UNIQUE	category_name	Category names must be distinct.
USERS	NOT NULL	role_id, full_name, email	Core user identification fields cannot be null.
EQUIPMENT	NOT NULL	equipment_name, category_id, laboratory_id, purchase_date, location	All descriptive and locational fields are mandatory for equipment.

Table Name	Key / Constraint	Attribute(s)	Business Rule Enforced
COMPONENTS	NOT NULL	component_name, category_id, total_quantity, available_quantity, minimum_stock	All quantity and identity fields are mandatory for components.
BOOKINGS	NOT NULL	equipment_id, user_id, booking_date, start_time, end_time, purpose	A booking requires complete time-slot and purpose information.
BREAKDOWNS	NOT NULL	equipment_id, reported_by, reported_date, problem_description	All breakdown report fields are mandatory.
MAINTENANCE_JOBS	NOT NULL	equipment_id, start_date, problem_description	Mandatory fields for a maintenance job.
COMPONENTS	CHECK	total_quantity >= 0	Total quantity cannot be a negative number.
COMPONENTS	CHECK	available_quantity >= 0	Available stock cannot be negative; prevents over-issuance.
COMPONENTS	CHECK	minimum_stock >= 0	Minimum stock threshold must be non-negative.
COMPONENTS	CHECK	unit_cost >= 0	Unit cost cannot be negative.
EQUIPMENT_ALLOCATION	CHECK	quantity > 0	Allocation quantity must be at least 1.
RETURNS	CHECK	returned_quantity > 0	Return quantity must be at least 1.
MAINTENANCE_JOBS	CHECK	cost >= 0	Maintenance cost cannot be negative.
EQUIPMENT	DEFAULT	status = 'Available'	Newly registered equipment starts in the Available state.
BOOKINGS	DEFAULT	status = 'PENDING'	All booking requests begin in PENDING state awaiting approval.
EQUIPMENT_ALLOCATION	DEFAULT	status = 'Issued'	A freshly created allocation is in Issued state.
NOTIFICATIONS	DEFAULT	is_read = FALSE	All new notifications are unread on creation.
USERS	DEFAULT	department = 'Robotics & Automation'	Default department for new user accounts.
EQUIPMENT	ENUM	status: Available, In Use, Maintenance, Decommissioned	Restricts equipment status to defined valid states.
BOOKINGS	ENUM	status: PENDING, APPROVED, REJECTED, IN_USE, RETURNED, CANCELLED	Restricts booking status lifecycle.
EQUIPMENT_ALLOCATION	ENUM	status: Issued, Returned, Partially Returned, Overdue, Lost	Restricts allocation status.
RETURNS	ENUM	condition_status: Good, Damaged, Burnt/Faulty, Missing Parts	Restricts the physical condition assessment at return.
BREAKDOWNS	ENUM	severity: Low, Medium, High, Critical	Classifies breakdown severity.
CALIBRATIONS	ENUM	status: Valid, Due Soon, Overdue	Tracks calibration validity states.

2.3 Functional Dependencies

The following functional dependencies (FDs) are defined for each relation in the final database. All FDs are derived directly from the implemented schema and domain semantics, using the standard notation of relational theory [1], [2].

```
ROLES:
    role_id → role_name, description, created_at

USERS:
    user_id → role_id, full_name, email, password_hash, roll_number, phone, department,
    created_at
    email → user_id          (alternate key)
    roll_number → user_id    (alternate key, partial - NULL values excluded)

LABORATORIES:
    lab_id → lab_name, location_code, incharge_id, contact_number

EQUIPMENT_CATEGORIES:
    category_id → category_name, description

EQUIPMENT:
    equipment_id → equipment_name, category_id, laboratory_id, serial_number,
    purchase_date, warranty_end, status, location, description, created_at
    serial_number → equipment_id    (alternate key)

COMPONENT_CATEGORIES:
    category_id → category_name, description

COMPONENTS:
    component_id → component_name, category_id, total_quantity, available_quantity,
    minimum_stock, unit_cost, storage_bin, specifications, created_at

PROJECTS:
    project_id → project_name, guide_faculty_id, start_date, end_date, status,
    description, created_at

PROJECT_MEMBERS:
    (project_id, user_id) → role_in_project, joined_date
    (No non-key attributes depend on a proper subset of the composite PK)

BOOKINGS:
    booking_id → equipment_id, user_id, project_id, booking_date, start_time,
    end_time, purpose, status, approved_by, created_at

EQUIPMENT_ALLOCATION:
    allocation_id → component_id, user_id, project_id, quantity, issue_date,
    expected_return_date, actual_return_date, status, remarks, created_at

USAGE_LOGS:
    log_id → equipment_id, user_id, project_id, start_timestamp, end_timestamp, log_notes

RETURNS:
    return_id → allocation_id, return_date, returned_quantity, condition_status,
    verified_by, penalty_or_notes

VENDORS:
    vendor_id → vendor_name, contact_person, phone, email, address, service_type,
    created_at

BREAKDOWNS:
```

```

breakdown_id → equipment_id, reported_by, reported_date, problem_description,
severity, status

MAINTENANCE_JOBS:
  job_id → equipment_id, breakdown_id, vendor_id, start_date, completion_date,
  cost, problem_description, action_taken, status

CALIBRATIONS:
  calibration_id → equipment_id, last_calibration_date, next_due_date, certified_by,
status, remarks

NOTIFICATIONS:
  notification_id → user_id, message, notification_type, is_read, created_at

REQUISITIONS:
  requisition_id → component_id, item_name, category_name, required_quantity,
  estimated_unit_cost, priority, status, vendor_name_1, buy_link_1,
  vendor_name_2, buy_link_2, vendor_name_3, buy_link_3, notes,
  requested_by, created_at

```

2.4 Normalization up to Third Normal Form (3NF)

Table 7 records the normal-form status of every relation, followed by a short discussion.

Table 7. Normalization Status up to 3NF

Relation Name	1NF	2NF	3NF	BCNF	Remarks / Decomposition
ROLES	Yes	Yes	Yes	Yes	Single-column PK; no multi-valued attributes; no partial or transitive dependencies.
USERS	Yes	Yes	Yes	Yes	Single-column PK. All non-key attributes (full_name, email, phone, department, etc.) depend directly and solely on user_id. The role_id FK does not introduce a transitive dependency on role attributes because role attributes reside in the separate ROLES table.
LABORATORIES	Yes	Yes	Yes	Yes	Single-column PK. incharge_id is an FK referencing USERS; incharge's name is not stored here, eliminating the transitive dependency that would exist in an unnormalized design.
EQUIPMENT_CATEGORIES	Yes	Yes	Yes	Yes	Lookup table; all attributes depend solely on category_id.
EQUIPMENT	Yes	Yes	Yes	Yes	Single-column PK (equipment_id). All attributes describe the specific equipment instance. category_name and lab_name are NOT stored in this table (they are resolved via FK JOIN), eliminating transitive dependencies present in an unnormalized design.
COMPONENT_CATEGORIES	Yes	Yes	Yes	Yes	Lookup table; all attributes depend solely on category_id.
COMPONENTS	Yes	Yes	Yes	Yes	Single-column PK. available_quantity is updated by triggers (not derived from other attributes in the table); category_name is not stored here. No transitive dependencies.
PROJECTS	Yes	Yes	Yes	Yes	Single-column PK. guide_faculty_id is FK; the guide's name is not stored, preventing transitive dependency.
PROJECT_MEMBERS	Yes	Yes	Yes	Yes	Composite PK (project_id, user_id). The only non-key attributes (role_in_project, joined_date) depend on the full composite key, not a subset. No partial dependencies exist.
BOOKINGS	Yes	Yes	Yes	Yes	Single-column PK. All booking attributes (date, time, purpose, status) describe a specific booking event, not the equipment or user independently. Student name, equipment name, and project name are not stored here.
EQUIPMENT_ALLOCATION	Yes	Yes	Yes	Yes	Single-column PK. All attributes (quantity, dates, status) describe the specific allocation transaction. Component name, student name, and project name are resolved via FK.
USAGE_LOGS	Yes	Yes	Yes	Yes	Single-column PK. start_timestamp and end_timestamp depend solely on log_id.

Relation Name	1NF	2NF	3NF	BCNF	Remarks / Decomposition
RETURNS	Yes	Yes	Yes	Yes	Single-column PK. All return-specific attributes (return_date, returned_quantity, condition_status) depend solely on return_id. Component and student details are accessible via allocation_id FK.
VENDORS	Yes	Yes	Yes	Yes	Single-column PK. No derived or repeating attributes; no transitive dependencies.
BREAKDOWNS	Yes	Yes	Yes	Yes	Single-column PK. breakdown_id determines all attributes of the failure report.
MAINTENANCE JOBS	Yes	Yes	Yes	Yes	Single-column PK. cost, action_taken, and status depend solely on job_id. vendor_name is not stored (accessed via vendor_id FK).
CALIBRATIONS	Yes	Yes	Yes	Yes	Single-column PK. All calibration schedule attributes depend solely on calibration_id.
NOTIFICATIONS	Yes	Yes	Yes	Yes	Single-column PK. message, notification_type, is_read, and created_at all depend on notification_id.
REQUISITIONS	Yes	Yes	Yes	Yes	Single-column PK. Multiple vendor columns (vendor_name_1, buy_link_1, etc.) are atomic fields, not repeating groups in the 1NF sense; they represent three distinct vendor options as separate attributes rather than multi-valued attributes. No partial or transitive dependencies exist.

Normalization Discussion:

All nineteen relations in the ROBO-LAB database satisfy First Normal Form (1NF) because every attribute in every relation contains only atomic values and every relation has a defined primary key. No multi-valued attributes or repeating groups are present. The normal-form definitions applied here follow standard database texts [2], [3].

Second Normal Form (2NF) is satisfied throughout the schema because only PROJECT_MEMBERS has a composite primary key, and the non-key attributes of that relation (role_in_project, joined_date) are fully functionally dependent on the entire composite key (project_id, user_id), not on a subset of it.

Third Normal Form (3NF) is satisfied because no non-key attribute in any relation is transitively dependent on the primary key through another non-key attribute. Descriptive attributes of referenced entities (such as equipment_name, student_full_name, category_name, vendor_name, and project_name) are never stored redundantly in child tables; they are consistently resolved through foreign key joins, which was the primary design strategy for eliminating transitive dependencies during schema construction.

Boyce-Codd Normal Form (BCNF) also holds for every relation. In each relation the only determinants are the primary key and the alternate keys (such as USERS.email, USERS.roll_number and EQUIPMENT.serial_number), and every alternate key is a candidate key [1]. No non-key attribute determines another attribute, so no relation requires further decomposition.

2.5 Data Dictionary

Table 8 gives the data dictionary for all 19 tables of the database.

Table 8. Data Dictionary

Table	Attribute	Data Type	Key / Constraint	Description
ROLES	role_id	INT AUTO INCREMENT	PK	Unique role identifier
	role_name	VARCHAR(50)	NOT NULL, UNIQUE	Role label (Admin, Lab Technician, Faculty, Student)
	description	VARCHAR(255)	—	Human-readable role description
	created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Timestamp of role record creation
USERS	user_id	INT AUTO INCREMENT	PK	Unique user identifier
	role_id	INT	FK → ROLES, NOT NULL	Assigned role of the user
	full_name	VARCHAR(100)	NOT NULL	User's full name
	email	VARCHAR(120)	NOT NULL, UNIQUE	Institutional email address (login credential)
	password_hash	VARCHAR(255)	DEFAULT	Hashed password string
	roll_number	VARCHAR(30)	UNIQUE	Student roll number or staff ID (nullable)
	phone	VARCHAR(20)	—	Contact phone number
	department	VARCHAR(80)	DEFAULT 'Robotics & Automation'	User's academic or staff department
	created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Account creation timestamp
LABORATORIES	lab_id	INT AUTO INCREMENT	PK	Unique laboratory identifier
	lab_name	VARCHAR(100)	NOT NULL, UNIQUE	Descriptive laboratory name
	location_code	VARCHAR(50)	NOT NULL	Room/block code for the laboratory
	incharge_id	INT	FK → USERS, ON DELETE SET NULL	User designated as laboratory in-charge
	contact_number	VARCHAR(20)	—	Direct phone number of the laboratory
EQUIPMENT_CATEGORIES	category_id	INT AUTO INCREMENT	PK	Unique equipment category identifier
	category_name	VARCHAR(100)	NOT NULL, UNIQUE	Category label (e.g., Robotic Arms & Manipulators)
	description	TEXT	—	Detailed category description
EQUIPMENT	equipment_id	VARCHAR(20)	PK	Unique asset ID (e.g., EQ001)
	equipment_name	VARCHAR(120)	NOT NULL	Descriptive name of the equipment
	category_id	INT	FK → EQUIPMENT_CATEGORIES, NOT NULL	Category classification
	laboratory_id	INT	FK → LABORATORIES, NOT NULL	Home laboratory of the equipment
	serial_number	VARCHAR(100)	UNIQUE	Manufacturer serial number

Table	Attribute	Data Type	Key / Constraint	Description
	purchase_date	DATE	NOT NULL	Date of procurement
	warranty_end	DATE	—	Warranty expiry date
	status	ENUM	DEFAULT 'Available'	Current status: Available, In Use, Maintenance, Decommissioned
	location	VARCHAR(100)	NOT NULL	Specific location within the laboratory
	description	TEXT	—	Technical specifications or description
	created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Record creation timestamp
COMPONENT_CATEGORIES	category_id	INT AUTO INCREMENT	PK	Unique component category identifier
	category_name	VARCHAR(100)	NOT NULL, UNIQUE	Category label (e.g., Microcontrollers & SBCs)
	description	TEXT	—	Category description
COMPONENTS	component_id	VARCHAR(20)	PK	Unique component identifier (e.g., C001)
	component_name	VARCHAR(120)	NOT NULL	Descriptive component name
	category_id	INT	FK → COMPONENT_CATEGORIES, NOT NULL	Category classification
	total_quantity	INT	NOT NULL, CHECK >= 0	Total units owned by the laboratory
	available_quantity	INT	NOT NULL, CHECK >= 0	Units currently available for issuance
	minimum_stock	INT	NOT NULL, DEFAULT 5, CHECK >= 0	Low-stock alert threshold
	unit_cost	DECIMAL(10,2)	DEFAULT 0.00, CHECK >= 0	Cost per unit in INR
	storage_bin	VARCHAR(50)	—	Physical storage bin code
	specifications	TEXT	—	Technical specifications
	created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Record creation timestamp
PROJECTS	project_id	INT AUTO INCREMENT	PK	Unique project identifier
	project_name	VARCHAR(150)	NOT NULL	Project title
	guide_faculty_id	INT	FK → USERS, ON DELETE SET NULL	Faculty guide for the project
	start_date	DATE	NOT NULL	Project start date
	end_date	DATE	—	Project end date (nullable if ongoing)
	status	ENUM	DEFAULT 'Active'	Project status: Active, Completed, On Hold
	description	TEXT	—	Project abstract/description
PROJECT_MEMBERS	created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Record creation timestamp
	project_id	INT	PK (composite), FK → PROJECTS	Associated project
	user_id	INT	PK (composite), FK → USERS	Associated student user
	role_in_project	VARCHAR(50)	DEFAULT 'Team Member'	Member's role within the project
BOOKINGS	joined_date	DATE	DEFAULT CURRENT_DATE	Date the member joined the project
	booking_id	INT AUTO INCREMENT	PK	Unique booking identifier
	equipment_id	VARCHAR(20)	FK → EQUIPMENT, NOT NULL	Equipment being booked
	user_id	INT	FK → USERS, NOT NULL	Student making the booking

Table	Attribute	Data Type	Key / Constraint	Description
	project_id	INT	FK → PROJECTS	Associated project (nullable)
	booking_date	DATE	NOT NULL	Date of the requested usage slot
	start time	TIME	NOT NULL	Slot start time
	end time	TIME	NOT NULL	Slot end time
	purpose	TEXT	NOT NULL	Stated purpose for the booking
	status	ENUM	DEFAULT 'PENDING'	Status: PENDING, APPROVED, REJECTED, IN_USE, RETURNED, CANCELLED
	approved_by	INT	FK → USERS, ON DELETE SET NULL	User (Faculty/Admin) who approved or rejected
	created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Request submission timestamp
EQUIPMENT_ALLOCATION	allocation_id	INT AUTO INCREMENT	PK	Unique allocation identifier
	component_id	VARCHAR(20)	FK → COMPONENTS, NOT NULL	Component being issued
	user_id	INT	FK → USERS, NOT NULL	Student receiving the component
	project_id	INT	FK → PROJECTS, NOT NULL	Project for which the component is issued
	quantity	INT	NOT NULL, CHECK > 0	Number of units issued
	issue_date	DATE	NOT NULL	Date of issuance
	expected_return_date	DATE	NOT NULL	Expected return deadline
	actual_return_date	DATE	—	Actual date returned (NULL if not yet returned)
	status	ENUM	DEFAULT 'Issued'	Status: Issued, Returned, Partially Returned, Overdue, Lost
	remarks	TEXT	—	Issuance remarks or notes
	created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Record creation timestamp
USAGE_LOGS	log_id	INT AUTO INCREMENT	PK	Unique log identifier
	equipment_id	VARCHAR(20)	FK → EQUIPMENT, NOT NULL	Equipment used in this session
	user_id	INT	FK → USERS, NOT NULL	User who operated the equipment
	project_id	INT	FK → PROJECTS, ON DELETE SET NULL	Associated project (nullable)
	start_timestamp	DATETIME	NOT NULL	Session start date and time
	end_timestamp	DATETIME	—	Session end date and time (NULL if session active)
	log_notes	TEXT	—	Technician or user notes about the session
RETURNS	return_id	INT AUTO INCREMENT	PK	Unique return record identifier
	allocation_id	INT	FK → EQUIPMENT_ALLOCATION, ON DELETE CASCADE	The allocation being returned
	return_date	DATE	NOT NULL	Date of return
	returned_quantity	INT	NOT NULL, CHECK > 0	Number of units physically returned
	condition_status	ENUM	DEFAULT 'Good'	Condition: Good, Damaged, Burnt/Faulty, Missing Parts
	verified_by	INT	FK → USERS, ON DELETE SET NULL	Staff member who verified the return

Table	Attribute	Data Type	Key / Constraint	Description
	penalty_or_notes	TEXT	—	Notes, damage descriptions, or penalty details
VENDORS	vendor_id	INT AUTO INCREMENT	PK	Unique vendor identifier
	vendor_name	VARCHAR(120)	NOT NULL	Name of the service vendor
	contact_person	VARCHAR(100)	—	Primary contact at the vendor
	phone	VARCHAR(25)	NOT NULL	Contact phone number
	email	VARCHAR(120)	—	Vendor email address
	address	TEXT	—	Physical address of the vendor
	service_type	VARCHAR(100)	—	Type of services provided
	created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Record creation timestamp
BREAKDOWNS	breakdown_id	INT AUTO INCREMENT	PK	Unique breakdown report identifier
	equipment_id	VARCHAR(20)	FK → EQUIPMENT, NOT NULL	Equipment experiencing the breakdown
	reported_by	INT	FK → USERS, NOT NULL	User who reported the breakdown
	reported_date	DATE	NOT NULL	Date the breakdown was reported
	problem_description	TEXT	NOT NULL	Detailed description of the failure
	severity	ENUM	DEFAULT 'Medium'	Severity: Low, Medium, High, Critical
	status	ENUM	DEFAULT 'Reported'	Status: Reported, Under Review, In Maintenance, Resolved
MAINTENANCE_JOBS	job_id	INT AUTO INCREMENT	PK	Unique maintenance job identifier
	equipment_id	VARCHAR(20)	FK → EQUIPMENT, NOT NULL	Equipment under maintenance
	breakdown_id	INT	FK → BREAKDOWNS, ON DELETE SET NULL	Related breakdown report (optional)
	vendor_id	INT	FK → VENDORS, ON DELETE SET NULL	Assigned service vendor (optional)
	start_date	DATE	NOT NULL	Maintenance start date
	completion_date	DATE	—	Maintenance completion date (NULL if in progress)
	cost	DECIMAL(10,2)	DEFAULT 0.00, CHECK >= 0	Maintenance service cost in INR
	problem_description	TEXT	NOT NULL	Description of the maintenance problem
	action_taken	TEXT	—	Actions performed during maintenance
	status	ENUM	DEFAULT 'Scheduled'	Status: Scheduled, In Progress, Completed, Cannot Repair
CALIBRATIONS	calibration_id	INT AUTO INCREMENT	PK	Unique calibration record identifier
	equipment_id	VARCHAR(20)	FK → EQUIPMENT, NOT NULL	Equipment calibrated
	last_calibration_date	DATE	NOT NULL	Date of most recent calibration
	next_due_date	DATE	NOT NULL	Date when next calibration is due
	certified_by	VARCHAR(120)	—	Certifying authority or person name
	status	ENUM	DEFAULT 'Valid'	Status: Valid, Due Soon, Overdue
	remarks	TEXT	—	Calibration findings and remarks

Table	Attribute	Data Type	Key / Constraint	Description
NOTIFICATIONS	notification_id	INT AUTO INCREMENT	PK	Unique notification identifier
	user_id	INT	FK → USERS, ON DELETE CASCADE, NOT NULL	Recipient user
	message	TEXT	NOT NULL	Notification message content
	notification_type	ENUM	DEFAULT 'Booking'	Type: Booking, Allocation, Return_Due, Breakdown, Maintenance, Stock Alert
	is_read	BOOLEAN	DEFAULT FALSE	Read/unread status
	created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Generation timestamp
REQUISITIONS	requisition_id	INT AUTO INCREMENT	PK	Unique requisition identifier
	component_id	VARCHAR(20)	FK → COMPONENTS, ON DELETE SET NULL	Linked component (optional)
	item_name	VARCHAR(150)	NOT NULL	Name of the item to be procured
	category_name	VARCHAR(100)	DEFAULT 'Electronics'	Category label for the item
	required_quantity	INT	NOT NULL, DEFAULT 1	Quantity to be ordered
	estimated_unit_cost	DECIMAL(10,2)	DEFAULT 0.00	Estimated per-unit cost in INR
	priority	ENUM	DEFAULT 'Medium'	Priority: Critical, High, Medium, Low
	status	ENUM	DEFAULT 'Pending Order'	Lifecycle: Pending Order, Approved, Ordered, In Transit, Received, Cancelled
	vendor_name_1	VARCHAR(100)	DEFAULT 'Robu.in'	First vendor option name
	buy_link_1	TEXT	—	Purchase URL for vendor 1
	vendor_name_2	VARCHAR(100)	DEFAULT 'ElectronicsComp'	Second vendor option name
	buy_link_2	TEXT	—	Purchase URL for vendor 2
	vendor_name_3	VARCHAR(100)	DEFAULT 'Amazon India'	Third vendor option name
	buy_link_3	TEXT	—	Purchase URL for vendor 3
	notes	TEXT	—	Additional procurement notes
	requested_by	INT	FK → USERS, ON DELETE SET NULL	User who submitted the requisition
	created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Submission timestamp

2.6 Implementation of Logical Design: DDL and Sample Data

Table 9 summarises the environment in which the logical design was implemented in MySQL [4].

Table 9. Database Implementation Environment

Item	Details
DBMS	MySQL
DBMS Version	MySQL 8.0
Database Name	robotics_lab

Item	Details
Number of Tables	19
Number of Views	5
Number of Triggers	3
Primary Storage Engine	InnoDB (supports foreign keys and ACID transactions)
Character Set	utf8mb4
Fallback DBMS	SQLite (used for local development via auto-fallback in config.py)
Sample Data	database/seed.sql populates all 19 tables, including 8 equipment records, 12 component types, 8 users, 4 projects, 5 bookings, 8 allocations, 3 vendors, 2 breakdowns, 2 maintenance jobs, 3 calibrations and 3 notifications.

Representative CREATE TABLE Statements:

The following excerpts illustrate key design decisions. The complete DDL script is provided in Appendix A.

```
-- Core asset table demonstrating VARCHAR primary key, ENUM status,
-- UNIQUE serial number, and two foreign key references
CREATE TABLE EQUIPMENT (
    equipment_id    VARCHAR(20) PRIMARY KEY,
    equipment_name  VARCHAR(120) NOT NULL,
    category_id     INT NOT NULL,
    laboratory_id   INT NOT NULL,
    serial_number   VARCHAR(100) UNIQUE,
    purchase_date   DATE NOT NULL,
    warranty_end    DATE,
    status          ENUM('Available','In
Use','Maintenance','Decommissioned')
                DEFAULT 'Available',
    location        VARCHAR(100) NOT NULL,
    description     TEXT,
    created_at      TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (category_id)
                REFERENCES EQUIPMENT_CATEGORIES(category_id) ON UPDATE CASCADE,
    FOREIGN KEY (laboratory_id)
                REFERENCES LABORATORIES(lab_id) ON UPDATE CASCADE
);

-- Quantity-tracked component table demonstrating CHECK constraints
CREATE TABLE COMPONENTS (
    component_id    VARCHAR(20) PRIMARY KEY,
    component_name  VARCHAR(120) NOT NULL,
    category_id     INT NOT NULL,
    total_quantity  INT NOT NULL CHECK (total_quantity >= 0),
    available_quantity INT NOT NULL CHECK (available_quantity >= 0),
    minimum_stock   INT NOT NULL DEFAULT 5 CHECK (minimum_stock >= 0),
    unit_cost       DECIMAL(10,2) DEFAULT 0.00 CHECK (unit_cost >= 0),
    storage_bin     VARCHAR(50),
    specifications  TEXT,
    created_at      TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (category_id)
                REFERENCES COMPONENT_CATEGORIES(category_id) ON UPDATE CASCADE
);

-- Trigger: Automatically deduct available quantity on component
allocation
CREATE TRIGGER trg_after_allocation_insert
AFTER INSERT ON EQUIPMENT_ALLOCATION
```

```

FOR EACH ROW
BEGIN
    UPDATE COMPONENTS
    SET available_quantity = available_quantity - NEW.quantity
    WHERE component_id = NEW.component_id;
END;

```

Representative Sample Records:

The seed data script (`database/seed.sql`) populates all 19 tables. Selected representative records are shown in Tables 10 to 12.

Table 10. Representative Records of the EQUIPMENT Table

equipment_id	equipment_name	status	laboratory_id	serial_number
EQ001	6-DOF Articulated Robotic Arm	Available	1	ARM-2024-X600
EQ002	Prusa MK4 3D Printer	Maintenance	3	PRUSA-MK4-9821
EQ003	Rigol 100MHz Digital Oscilloscope	Available	1	RIGOL-DS1104Z
EQ004	TurtleBot4 ROS2 Mobile Robot	In Use	2	TB4-NAV-0042

Table 11. Representative Records of the COMPONENTS Table

component_id	component_name	total_qty	available_qty	minimum_stock	unit_cost
C001	Arduino UNO R3	35	20	8	450.00
C002	ESP32 Dual-Core DevKit V1	30	14	6	380.00
C005	MPU-6050 6-DOF IMU	25	8	8	160.00
C010	OAK-D Spatial AI Camera Module	5	1	2	18500.00

Table 12. Representative Records of the EQUIPMENT_ALLOCATION Table

allocation_id	component_id	user_id	project_id	quantity	issue_date	expected_return	status
1	C001	5	1	1	2026-08-15	2026-09-15	Issued
5	C003	7	3	1	2026-08-20	2026-09-20	Issued
8	C005	6	1	1	2026-08-10	2026-08-24	Returned

CHAPTER 3

APPLICATION DEVELOPMENT USING LOGICAL DATABASE DESIGN

3.1 Application Overview

ROBO-LAB is a full-stack web application built using Python [5] with the Flask framework [6] as the backend and HTML/CSS/JavaScript as the frontend. The application implements a role-based resource management system for a college robotics laboratory, serving four distinct user roles: Admin, Lab Technician, Faculty, and Student. Each role is presented with a tailored interface that exposes only the functionality appropriate to that role's responsibilities.

The application communicates with MySQL through the `mysql-connector-python` library [7] using a centralized database helper module (`db.py`). All database interactions are performed using parameterized queries, which prevent SQL injection attacks. The application includes an automatic fallback mechanism: if the MySQL server is unavailable, the application transparently falls back to a SQLite database file (`robolab.db`), enabling local development and demonstration without a running MySQL instance.

The Flask application defines approximately 40 routes organized into functional modules corresponding directly to the database tables. The `app.py` file contains all route handlers; templates are Jinja2 HTML files in the `templates/` directory; static assets including JavaScript and CSS are in the `static/` directory. A context processor (`inject_global_metrics`) injects the current user's role, unread notification count, pending booking count, and low-stock alert count into every template at render time.

The application demonstrates the practical use of the relational database design, including multi-table JOIN queries for displaying related data, trigger-driven stock management, view-based report queries, and full CRUD operations across all 19 tables.

3.2 Development Environment and Database Connectivity

The tools and technologies used to build the application are listed in Table 13.

Table 13. Application Development Environment

Item	Details
Programming Language	Python
Backend Framework	Flask 3.0.3
Frontend	HTML5, CSS3, JavaScript

Item	Details
Database	MySQL
Database Connector	mysql-connector-python 8.4.0
Environment Variables	python-dotenv 1.0.1
IDE	VS Code
Operating System	Windows
MySQL Host	localhost (default), configurable via .env
MySQL Port	3306 (default)
Database Name	robotics_lab
Development Fallback DB	SQLite (robolab.db via auto-fallback)
Application Entry Point	app.py (Flask dev server on port 5000)
Repository	https://github.com/MSN-2007/Lab_Management-.git

Database Connectivity (representative code from db.py / config.py):

```
# config.py - Database connection configuration
import os
from dotenv import load_dotenv
load_dotenv()

class Config:
    DB_HOST = os.getenv('DB_HOST', 'localhost')
    DB_PORT = int(os.getenv('DB_PORT', 3306))
    DB_USER = os.getenv('DB_USER', 'root')
    DB_PASSWORD = os.getenv('DB_PASSWORD', 'root')
    DB_NAME = os.getenv('DB_NAME', 'robotics_lab')
    AUTO_FALLBACK_SQLITE = os.getenv('AUTO_FALLBACK_SQLITE', 'True') == 'True'
    SQLITE_DB_PATH = os.path.join(os.path.dirname(
        os.path.abspath(__file__)), 'database', 'robolab.db')
```

3.3 Application Modules and Database Mapping

Table 14 maps each application module to the tables it uses and the main database operations it performs.

Table 14. Application Module-to-Database Mapping

Module / Screen	Purpose	Tables Used	Main Operations
Login & Registration	Authenticate users; register new accounts	USERS, ROLES	SELECT (credential check), INSERT (register), SESSION management
Dashboard	Role-tailored overview with live statistics and recent activity feeds	EQUIPMENT, COMPONENTS, EQUIPMENT_ALLOCATION, BOOKINGS, MAINTENANCE_JOBS, PROJECTS, NOTIFICATIONS	SELECT COUNT(*), multi-table JOINS, conditional queries by user role
Equipment Catalog	Display, search, filter, add, and update equipment	EQUIPMENT, EQUIPMENT_CATEGORIES, LABORATORIES	SELECT with JOIN + WHERE, INSERT, UPDATE (status)
Equipment Detail & Timeline	Per-equipment event history (bookings, breakdowns, maintenance, calibrations)	EQUIPMENT, BOOKINGS, BREAKDOWNS, MAINTENANCE_JOBS, CALIBRATIONS, USERS, VENDORS	Multi-table SELECT, UNION-based timeline aggregation
Component Inventory	View, add, and restock components; low-stock filtering	COMPONENTS, COMPONENT_CATEGORIES	SELECT with JOIN + WHERE, INSERT, UPDATE (quantity)

Module / Screen	Purpose	Tables Used	Main Operations
Projects & Teams	Manage academic projects and project member assignments	PROJECTS, PROJECT_MEMBERS, USERS	SELECT with GROUP BY + COUNT, INSERT, JOIN
Bookings & Approvals	Submit and manage equipment booking requests; approve/reject	BOOKINGS, EQUIPMENT, USERS, PROJECTS	SELECT with multi-JOIN, INSERT, UPDATE (status, approved_by)
Allocations (Issue/Return)	Issue components to students; record returns with condition	EQUIPMENT_ALLOCATION, COMPONENTS, USERS, PROJECTS, RETURNS	SELECT with JOIN, INSERT allocation, INSERT return, UPDATE status, trigger-driven UPDATE of COMPONENTS
Usage Logs	View session logs for equipment usage	USAGE_LOGS, EQUIPMENT, USERS, PROJECTS	SELECT with JOIN, INSERT (auto on booking start)
Maintenance & Breakdowns	Report breakdowns; create and complete maintenance jobs	BREAKDOWNS, MAINTENANCE_JOBS, EQUIPMENT, VENDORS, USERS	SELECT with JOIN, INSERT, UPDATE (status, completion_date, action_taken)
Calibrations	View calibration schedules and due-date statuses	CALIBRATIONS, EQUIPMENT	SELECT with JOIN, INSERT
Vendors	View vendors with service job count and total billed amount	VENDORS, MAINTENANCE_JOBS	SELECT with LEFT JOIN, GROUP BY, SUM, INSERT
Notifications	View and manage system notifications	NOTIFICATIONS, USERS	SELECT, UPDATE (is_read)
Reports Playground	Execute pre-defined SQL analytical reports	All relevant tables	Complex SELECT, JOIN, LEFT JOIN, GROUP BY, HAVING, ORDER BY, UNION, Subquery
Procurement / Requisitions	Submit and manage component purchase orders	REQUISITIONS, COMPONENTS, USERS	SELECT with LEFT JOIN, INSERT, UPDATE (status), UPDATE COMPONENTS on receipt

3.4 CRUD Operations

The Create, Read, Update and Delete operations supported by the application are summarised in Table 15.

Table 15. CRUD Operations

Operation	Application Feature / Screen	Table(s) Affected	Evidence / Route
Create	Register new user account	USERS	POST /register, POST /users/add
	Add new laboratory	LABORATORIES	NA
	Register equipment asset	EQUIPMENT	POST /equipment/add
	Add component to inventory	COMPONENTS	POST /components/add
	Create project	PROJECTS	POST /projects/create
	Submit equipment booking request	BOOKINGS	POST /bookings/create
	Issue component allocation	EQUIPMENT_ALLOCATION; COMPONENTS (trigger)	POST /allocations/issue
	Record component return	RETURNS; EQUIPMENT_ALLOCATION; COMPONENTS	POST /allocations/<id>/return
	Report equipment breakdown	BREAKDOWNS; EQUIPMENT (status update)	POST /breakdowns/report
	Create maintenance job	MAINTENANCE_JOBS; EQUIPMENT; BREAKDOWNS	POST /maintenance/create
	Add calibration record	CALIBRATIONS	POST /calibrations (form)
	Add vendor	VENDORS	POST /vendors (form)
	Submit procurement requisition	REQUISITIONS	POST /procurement/add

Operation	Application Feature / Screen	Table(s) Affected	Evidence / Route
Read	Search and filter equipment	EQUIPMENT, EQUIPMENT_CATEGORIES, LABORATORIES	GET /equipment?status=&category=&q=
	View component inventory with low-stock filter	COMPONENTS, COMPONENT_CATEGORIES	GET /components?low_stock=1
	View allocation list with status filter	EQUIPMENT_ALLOCATION, COMPONENTS, USERS, PROJECTS	GET /allocations?status=Issued
	View bookings with status filter	BOOKINGS, EQUIPMENT, USERS, PROJECTS	GET /bookings?status=PENDING
	View equipment timeline history	EQUIPMENT, BOOKINGS, BREAKDOWNS, MAINTENANCE_JOBS, CALIBRATIONS	GET /equipment/<id>
	View dashboard metrics	Multiple tables	GET /
	Execute report queries	Multiple tables	GET /reports?report=<key>
Update	Edit user account details	USERS	POST /users/edit/<id>
	Update equipment status	EQUIPMENT	POST /equipment/<id>/status
	Approve / Reject booking	BOOKINGS	POST /bookings/<id>/status (action=APPROVE/REJECT)
	Start equipment usage session	BOOKINGS, EQUIPMENT, USAGE_LOGS	POST /bookings/<id>/status (action=START_USE)
	Complete equipment usage session	BOOKINGS, EQUIPMENT	POST /bookings/<id>/status (action=COMPLETE)
	Complete maintenance job	MAINTENANCE_JOBS, EQUIPMENT, BREAKDOWNS	POST /maintenance/<id>/complete
	Update procurement requisition status	REQUISITIONS; COMPONENTS (on Received)	POST /procurement/status/<id>
	Restock component inventory	COMPONENTS	POST /components/restock
	Update user profile	USERS	POST /profile
Delete	Delete user account	USERS	POST /users/delete/<id> (with self-deletion guard)
	Delete procurement requisition	REQUISITIONS	POST /procurement/delete/<id>

3.5 Business Rules, Validation and Error Handling

Table 16 lists the business rules enforced by the application and the database, and how each is implemented.

Table 16. Business Rules, Validation and Error Handling

Rule / Validation	How Implemented	Evidence / Implementation Location
BR-01 Component available quantity must not become negative on allocation.	Pre-check: if <code>comp['available_quantity'] < qty:</code> <code>flash(error)</code> before INSERT. Server-side validation prevents insertion.	<code>allocation_issue()</code> route in <code>app.py</code>
BR-02 Returned quantity must be positive.	CHECK constraint: <code>returned_quantity > 0</code> enforced at database level.	RETURNS table
BR-03 Allocation quantity must be positive.	CHECK constraint: <code>quantity > 0</code> enforced at database level.	EQUIPMENT_ALLOCATION table
BR-04 Equipment with High/Critical severity breakdown is automatically set to Maintenance status.	Application-level: if severity in <code>['High', 'Critical']:</code> <code>execute_db(UPDATE EQUIPMENT SET status='Maintenance')</code> . Database trigger provides same logic at DB layer.	<code>breakdown_report()</code> route; <code>trg_after_breakdown_insert</code> trigger

Rule / Validation	How Implemented	Evidence / Implementation Location
BR-05 Maintenance job creation sets equipment status to Maintenance.	<code>execute_db("UPDATE EQUIPMENT SET status='Maintenance'...")</code> executed after INSERT on MAINTENANCE_JOBS.	<code>maintenance_create()</code> route
BR-06 Maintenance completion restores equipment to Available status and resolves the linked breakdown.	<code>execute_db("UPDATE EQUIPMENT SET status='Available'...")</code> and <code>UPDATE BREAKDOWNS SET status='Resolved'</code> on job completion.	<code>maintenance_complete()</code> route
BR-07 Component stock is automatically decremented on allocation.	MySQL AFTER INSERT trigger on EQUIPMENT_ALLOCATION updates COMPONENTS.available_quantity. Application also performs an explicit UPDATE for SQLite compatibility.	<code>trg_after_allocation_insert</code> trigger
BR-08 Component stock is conditionally restored on return (only for Good or Damaged condition; not for Burnt/Faulty or Missing Parts).	Trigger: IF NEW.condition_status IN ('Good', 'Damaged') THEN UPDATE COMPONENTS.... Application mirrors this logic.	<code>trg_after_return_insert</code> trigger; <code>allocation_return()</code> route
BR-09 A user cannot delete their own active account.	Guard: if <code>curr == user_id</code> : <code>flash(warning); return redirect.</code>	<code>users_delete()</code> route
BR-10 Only authorized roles (not Student) can register new equipment.	Role check: if <code>user['role_name'] == 'Student'</code> : <code>flash(Unauthorized).</code>	<code>equipment_add()</code> route
BR-11 All SQL queries use parameterized query syntax.	<code>mysql-connector-python</code> parameterized queries: <code>%s</code> placeholders with separate parameter lists prevent SQL injection.	All routes in <code>app.py</code> using <code>query_db()</code> and <code>execute_db()</code>
BR-12 All form submissions validate required fields before database insertion.	if not <code>full_name</code> or not <code>email</code> : <code>flash(error); return redirect</code> pattern used consistently.	All POST routes
BR-13 All database operations are wrapped in try/except blocks with user-facing error messages.	try: <code>execute_db(...)</code> except Exception as <code>e</code> : <code>flash(f"Error: {str(e)}", 'danger')</code> pattern used throughout.	All POST routes in <code>app.py</code>
BR-14 Low-stock alerts are computed dynamically on each page load for the navigation badge.	<code>SELECT COUNT(*) FROM COMPONENTS WHERE available_quantity <= minimum_stock</code> executed on every request.	<code>inject_global_metrics()</code> context processor
BR-15 Unique email and roll number constraints prevent duplicate accounts.	Database UNIQUE constraint returns an exception caught by the try/except handler; pre-check query also validates before INSERT.	USERS table (UNIQUE constraints); <code>register()</code> route
BR-16 Cost and quantity fields enforce non-negative values.	<code>CHECK (cost >= 0), CHECK (unit_cost >= 0), CHECK (total_quantity >= 0)</code> in DDL.	Database CHECK constraints
BR-17 Booking status follows a defined lifecycle (PENDING → APPROVED/REJECTED → IN USE → RETURNED).	Action-based status transitions via the action form parameter (APPROVE, REJECT, START_USE, COMPLETE).	<code>booking_update_status()</code> route
BR-18 When a requisition is marked as Received, component stock is automatically updated.	if <code>new_status == 'Received'</code> and <code>r['component_id']</code> : <code>UPDATE COMPONENTS SET total_quantity = total_quantity + required_quantity....</code>	<code>procurement_update_status()</code> route

3.6 Reports and Database Operations

The analytical reports and the SQL concepts they demonstrate are listed in Table 17.

Table 17. Reports and Database Operations

Report / Operation	SQL / Database Concept Used	Application Output
R-01 Equipment Availability Report	SELECT, INNER JOIN (3 tables), WHERE, ORDER BY	Retrieves all equipment with status='Available' joined with their categories and laboratories. Corresponds to SQL View <code>vw_available_equipment</code> .

Report / Operation	SQL / Database Concept Used	Application Output
R-02 Current Component Allocations to Students	SELECT, 5-table INNER JOIN (EQUIPMENT_ALLOCATION, USERS, COMPONENTS, COMPONENT_CATEGORIES, PROJECTS), WHERE, ORDER BY	Answers "who has which component for which project" – the primary operational query of the system.
R-03 Low Stock & Inventory Shortage Report	SELECT, INNER JOIN, WHERE (available_quantity <= minimum_stock), computed column (shortage_units), ORDER BY	Identifies components below safety stock threshold. Corresponds to SQL View vw_low_stock_components.
R-04 Equipment Maintenance & Repair History	SELECT, INNER JOIN (MAINTENANCE_JOBS, EQUIPMENT), LEFT JOIN (VENDORS), COALESCE for NULL vendor, ORDER BY	Shows full maintenance log. LEFT JOIN used because vendor is optional. Corresponds to SQL View vw_equipment_maintenance_history.
R-05 Equipment Booking Frequency & Utilization	SELECT, INNER JOIN (EQUIPMENT, EQUIPMENT_CATEGORIES), LEFT JOIN (BOOKINGS), COUNT, conditional SUM (CASE WHEN), GROUP BY, ORDER BY	Calculates total and successful bookings per equipment to assess utilization.
R-06 Total Component Value Issued per Project	SELECT, multi-table INNER JOIN, COUNT, SUM (quantity × unit_cost), GROUP BY, ORDER BY	Computes total INR value of components issued to each project.
R-07 Equipment Breakdown Frequency	SELECT, INNER JOIN (EQUIPMENT, BREAKDOWNS), COUNT, MAX (last_breakdown_date), GROUP BY, ORDER BY	Identifies most frequently broken equipment.
R-08 Calibration Schedules & Due Date Audit	SELECT, INNER JOIN (CALIBRATIONS, EQUIPMENT), DATEDIFF, ORDER BY next due date	Audit of calibration validity states for measurement instruments.
R-09 Equipment Timeline Audit (per equipment)	SELECT, UNION ALL (3 subqueries: BOOKINGS, BREAKDOWNS, MAINTENANCE_JOBS), JOIN, COALESCE, ORDER BY	Produces a chronological event history for a specific equipment unit (EQ001 demonstrated in queries.sql).
R-10 Overdue Component Returns (Subquery)	SELECT (USERS), WHERE user_id IN (SELECT from EQUIPMENT_ALLOCATION WHERE status='Issued' AND expected_return_date < CURRENT_DATE)	Identifies students with overdue allocations using a correlated subquery.
R-11 Equipment Never Broken (NOT IN Subquery)	SELECT (EQUIPMENT), WHERE equipment_id NOT IN (SELECT DISTINCT equipment_id FROM BREAKDOWNS)	Identifies equipment with a clean breakdown record using a NOT IN subquery.
R-12 Vendor Service Summary	SELECT VENDORS, LEFT JOIN MAINTENANCE_JOBS, COUNT(job_id), SUM(cost), GROUP BY vendor_id	Displayed on the Vendors page; aggregates service job count and total billed amount per vendor.

All reports are implemented as pre-defined queries in the Reports Playground module (/reports route in app.py). Reports R-01 through R-08 are available via the interactive

reports dropdown. Reports R-09 through R-11 are demonstrated in `database/queries.sql` and are available for execution during the academic viva review.

3.7 Application Screens

Figures 2 to 9 show the key screens of the application running on the seeded database.

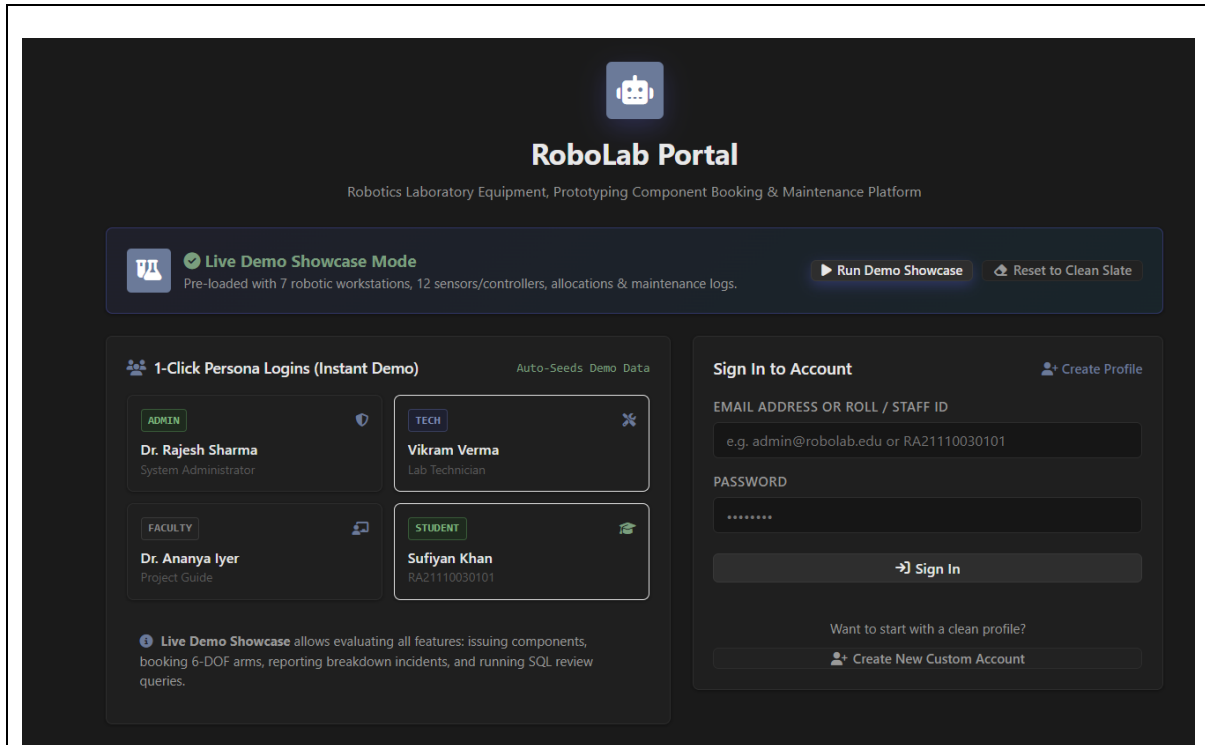


Figure 2. Login Screen

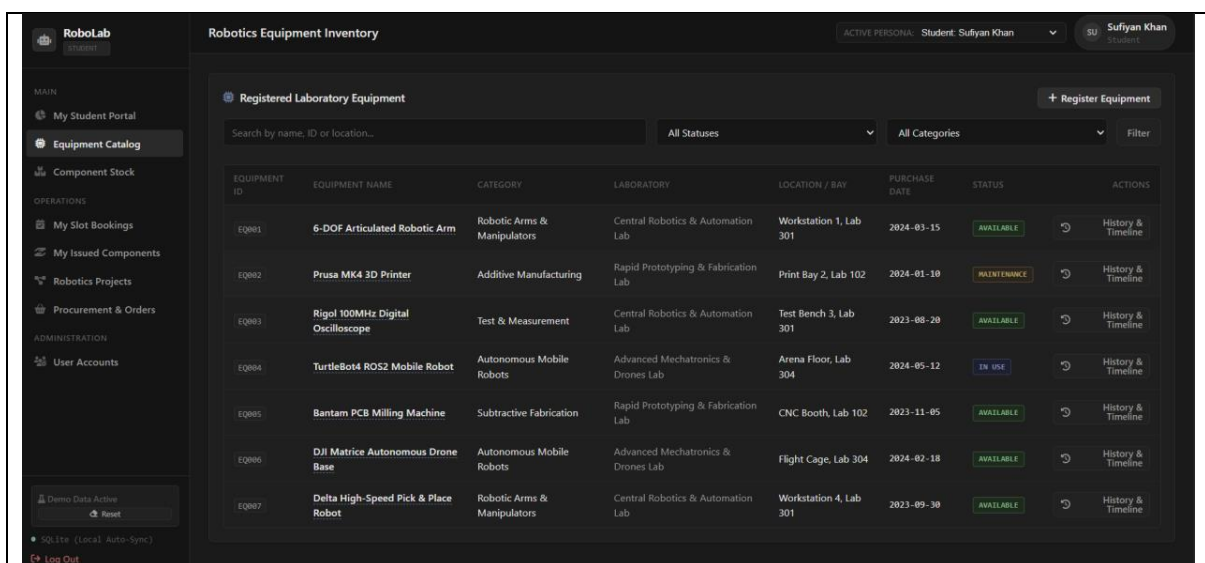


Figure 3. Role-Tailored Dashboard

RoboLab

ADMIN

MAIN

Dashboard

Equipment Catalog

Component Stock 3

OPERATIONS

Equipment Bookings 1

Project Allocations

Robotics Projects

Procurement & Orders

ADMINISTRATION

User Accounts

Maintenance

Calibrations

Vendors

Demo Data Active

Reset

SQLite (Local Auto-Sync)

Log Out

Robotics Equipment Inventory

ACTIVE PERSONA: Admin: Dr. Rajesh Sharma

DR. Dr. Rajesh Sharma

Registered Laboratory Equipment

+ Register Equipment

Search by name, ID or location...

All Statuses

All Categories

Filter

EQUIPMENT ID	EQUIPMENT NAME	CATEGORY	LABORATORY	LOCATION / BAY	PURCHASE DATE	STATUS	ACTIONS
EQ001	6-DOF Articulated Robotic Arm	Robotic Arms & Manipulators	Central Robotics & Automation Lab	Workstation 1, Lab 301	2024-03-15	AVAILABLE	History & Timeline
EQ002	Prusa MK4 3D Printer	Additive Manufacturing	Rapid Prototyping & Fabrication Lab	Print Bay 2, Lab 102	2024-01-10	MAINTENANCE	History & Timeline
EQ003	Rigol 100MHz Digital Oscilloscope	Test & Measurement	Central Robotics & Automation Lab	Test Bench 3, Lab 301	2023-08-20	AVAILABLE	History & Timeline
EQ004	TurtleBot4 ROS2 Mobile Robot	Autonomous Mobile Robots	Advanced Mechatronics & Drones Lab	Arena Floor, Lab 304	2024-05-12	IN USE	History & Timeline
EQ005	Bantam PCB Milling Machine	Subtractive Fabrication	Rapid Prototyping & Fabrication Lab	CNC Booth, Lab 102	2023-11-05	AVAILABLE	History & Timeline
EQ006	DJI Matrice Autonomous Drone Base	Autonomous Mobile Robots	Advanced Mechatronics & Drones Lab	Flight Cage, Lab 304	2024-02-18	AVAILABLE	History & Timeline
EQ007	Delta High-Speed Pick & Place Robot	Robotic Arms & Manipulators	Central Robotics & Automation Lab	Workstation 4, Lab 301	2023-09-30	AVAILABLE	History & Timeline

Figure 4. Equipment Catalog with Status Filtering

RoboLab ADMIN MAIN Dashboard Equipment Catalog Component Stock 3 OPERATIONS Equipment Bookings 1 Project Allocations Robotics Projects Procurement & Orders ADMINISTRATION User Accounts Maintenance Calibrations Vendors Demo Data Active Reset SQLite (Local Auto-Sync) Log Out	Procurement & Module Reorder List							ACTIVE PERSONA: Admin: Dr. Rajesh Sharma	DR. Dr. Rajesh Sharma
	All Orders Pending Approval Ordered In Transit Received & Stocked Search item or store...								
	PRIORITY	ITEM / MODULE NAME	CATEGORY	REQUIRED QTY	EST. COST	VENDOR BUYING LINKS (2-3 LINKS)		STATUS	ACTION
	CRITICAL	OAK-D Spatial AI Camera Module Depth sensing for TurtleBot4 autonomous delivery navigators. By Prof. Anind Menon	Sensors	2 units	₹37000.00 @ ₹18500.00/ea	Robu.in ElectronicsComp Amazon India		PENDING	Pending
	CRITICAL	MG996R Metal Gear High Torque Servo Replacement servos for 6-DOF Robotic Arm & Grippers. By Vikram Verma	Motors & Actuators	12 units	₹4080.00 @ ₹340.00/ea	Robu.in Robocraze Amazon India		PENDING	Pending
	HIGH	Arduino UNO R3 Auto-generated restock request for low stock (20 units remaining). By Safyan Khan	Electronics Restock	5 units	₹2250.00 @ ₹450.00/ea	Robu.in ElectronicsComp Amazon India		PENDING	Pending
	HIGH	Raspberry Pi 4 Model B (8GB RAM Edition) New edge computing modules for ROS2 stack navigation. By Dr. Ananya Iyer	Microcontrollers & SBCs	2 units	₹13600.00 @ ₹6800.00/ea	Robu.in ElectronicsComp Amazon India		PENDING	Pending
	HIGH	L298N Dual H-Bridge Motor Driver Module Replacement drivers for line follower rovers. By Vikram Verma	Power & Motor Drivers	8 units	₹960.00 @ ₹120.00/ea	Robu.in ElectronicsComp Amazon India		IN TRANSIT	In Transit
		ESP32 Dual-Core DevKit V1							

Figure 5. Component Inventory and Stock Management

Equipment Booking & Slot Management

ACTIVE PERSONA:

Admin: Dr. Rajesh Sharma

DR.

Dr. Rajesh Sharma

Admin

Equipment Slot Reservations

+ New Booking Request

All Bookings

Pending Approvals

Approved Slots

In Session (Active)

Completed & Returned

BOOKING ID	EQUIPMENT	STUDENT / RESEARCHER	PROJECT	SLOT DATE & TIME	PURPOSE / EXPERIMENT	STATUS	WORKFLOW ACTION
#3	Rigol 100MHz Digital Oscilloscope Test Bench 3, Lab 301	Pooja Nair RA21110030102	Autonomous Line & Maze Following Robot	2026-09-02 11:00:00 - 13:00:00	PWM frequency and motor back-EMF analysis on oscilloscope	PENDING	Approve Reject
#2	TurtleBot4 ROS2 Mobile Robot Arena Floor, Lab 304	Rahul Sundaram RA21110030103	Autonomous Mobile Delivery Robot (ROS2)	2026-09-01 14:00:00 - 17:00:00	SLAM map generation in lab corridor using LiDAR	IN USE	✓ Mark Returned
#1	6-DOF Articulated Robotic Arm Workstation 1, Lab 301	Sufiyan Khan RA21110030101	Vision Guided 6-DOF Pick & Place Robotic Arm	2026-09-01 10:00:00 - 12:00:00	Inverse kinematics calibration and gripper testing with vision pipeline	APPROVED	▶ Start Session
#5	Bantam PCB Milling Machine CNC Booth, Lab 102	Pooja Nair RA21110030102	Autonomous Line & Maze Following Robot	2026-08-28 14:00:00 - 16:00:00	Milling motor driver custom shield PCB	RETURNED	—
#4	6-DOF Articulated Robotic Arm Workstation 1, Lab 301	Rahul Sundaram RA21110030103	Vision Guided 6-DOF Pick & Place Robotic Arm	2026-08-25 09:00:00 - 11:30:00	Repeatability and precision test for pick and place	RETURNED	—

Figure 6. Booking Management and Approval Screen

Component Issue & Allocation Tracking

ACTIVE PERSONA:

Admin: Dr. Rajesh Sharma

▼

DR

Dr. Rajesh Sharma

Admin

All Allocations

Currently Issued (Active)

Returned

ALLOC ID	STUDENT BORROWER	COMPONENT DETAILS	PROJECT	QTY ISSUED	ISSUE DATE	DUE RETURN DATE	STATUS	RETURN VERIFICATION
#8	Pooja Nair RA21110030102	MPU-6050 6-DOF IMU (Gyro + Accel) C005 Bin: BIN-S2	Autonomous Line & Maze Following Robot	1	2026-08-10	2026-08-24	RETURNED	Returned: 2026-08-24
#7	Sneha Patel RA21110030104	ESP32 Dual-Core DevKit V1 C002 Bin: BIN-A2	Quadrotor Drone Indoor Obstacle Avoidance	2	2026-08-22	2026-09-05	ISSUED	🔄 Process Return
#6	Rahul Sundaram RA21110030103	RPLIDAR A1M8 360 Degree Laser Scanner C006 Bin: BIN-S3	Autonomous Mobile Delivery Robot (ROS2)	1	2026-08-20	2026-09-20	ISSUED	🔄 Process Return
#5	Rahul Sundaram RA21110030103	Raspberry Pi 4 Model B (4GB) C003 Bin: BIN-A3	Autonomous Mobile Delivery Robot (ROS2)	1	2026-08-20	2026-09-20	ISSUED	🔄 Process Return
#4	Sufiyan Khan RA21110030101	L298N Dual H-Bridge Motor Driver C009 Bin: BIN-P1	Autonomous Line & Maze Following Robot	1	2026-08-15	2026-09-15	ISSUED	🔄 Process Return
#3	Sufiyan Khan RA21110030101	MG996R High Torque Metal Gear Servo C007 Bin: BIN-M1	Autonomous Line & Maze Following Robot	2	2026-08-15	2026-09-15	ISSUED	🔄 Process Return
#2	Sufiyan Khan RA21110030101	HC-SR04 Ultrasonic Distance Sensor C004 Bin: BIN-S1	Autonomous Line & Maze Following Robot	2	2026-08-15	2026-09-15	ISSUED	🔄 Process Return
#1	Sufiyan Khan RA21110030101	Arduino UNO R3 C001 Bin: BIN-A1	Autonomous Line & Maze Following Robot	1	2026-08-15	2026-09-15	ISSUED	🔄 Process Return

Figure 7. Component Allocation – Issue and Return

Equipment Maintenance & Incident Management

ACTIVE PERSONA: Admin: Dr. Rajesh Sharma

DR. Dr. Rajesh SharmaAdmin

Report Breakdown Incident

Launch Maintenance Job

Active & Completed Maintenance Jobs

JOB ID	EQUIPMENT	PROBLEM SUMMARY	ACTION TAKEN	ASSIGNED VENDOR	START DATE	COMPLETED DATE	COST (₹)	STATUS	ACTION
#2	Prusa MK4 3D Printer	Extruder thermal runaway and thermistor open-circuit	Thermistor replaced, firmware PID autotuning in progress	3D Maker Solutions Ltd.	2026-08-30	‐	₹1500.00	IN PROGRESS	Complete
#1	6-DOF Articulated Robotic Arm	Servo jitter on Joint 3 gear set	Replaced feedback potentiometer and re-calibrated harmonic gear backlash.	RoboTech Automation & Spares	2026-08-11	2026-08-13	₹2400.00	COMPLETED	Closed

Reported Equipment Breakdowns

BREAKDOWN ID	EQUIPMENT	REPORTED BY	INCIDENT DATE	PROBLEM DESCRIPTION	SEVERITY	STATUS
#2	6-DOF Articulated Robotic Arm	Sufiyan Khan	2026-08-10	Joint 3 servo jitter and minor angle offset during payload lift.	MEDIUM	RESOLVED
#1	Prusa MK4 3D Printer	Pooja Nair	2026-08-29	Hotend heating failure and thermal runaway error on extruder 1.	HIGH	IN MAINTENANCE

Figure 8. Maintenance and Breakdown Management

Frequent Equipment Breakdowns by Severity				2 ROWS RETURNED
Shows aggregate breakdown counts and highest severity incidents per equipment.				
<pre> SELECT e.equipment_id, e.equipment_name, COUNT(brk.breakdown_id) AS total_breakdowns, MAX(brk.reported_date) AS last_breakdown_date FROM EQUIPMENT e JOIN BREAKDOWNS brk ON e.equipment_id = brk.equipment_id GROUP BY e.equipment_id, e.equipment_name ORDER BY total_breakdowns DESC; </pre>				Executed SQL Statement
EQUIPMENT ID	EQUIPMENT NAME	TOTAL BREAKDOWNS	LAST BREAKDOWN DATE	
EQ001	6-DOF Articulated Robotic Arm	1	2026-08-10	
EQ002	Prusa MK4 3D Printer	1	2026-08-29	

Figure 9. Reports Playground – SQL Output View

3.8 Testing and Results

Table 18 lists the positive and negative test cases designed to verify the application against the database design.

Table 18. Test Cases

Test ID	Feature	Input / Action	Expected Result	Actual Result	Status
TC-01	Equipment registration with valid data	Valid equipment_id, name, category, lab, serial number, purchase date, location via POST /equipment/add	Record created in EQUIPMENT table; success flash message displayed	Equipment record was successfully inserted into the EQUIPMENT table and the success message was displayed.	Pass
TC-02	Duplicate serial number rejection	Submit equipment with serial_number that already exists in EQUIPMENT table	MySQL UNIQUE constraint violation; error flash message displayed; no duplicate record inserted	Duplicate serial number was rejected, an error message was displayed, and no duplicate equipment record was created.	Pass

Test ID	Feature	Input / Action	Expected Result	Actual Result	Status
TC-03	Component allocation with sufficient stock	Allocate 2 units of Arduino UNO R3 (C001, available_qty = 20)	Allocation record inserted; COMPONENTS.available_quantity decremented to 18; success flash message	Allocation was successfully created and available quantity decreased from 20 to 18. A success message was displayed.	Pass
TC-04	Component allocation exceeding available stock	Attempt to allocate 25 units of Arduino UNO R3 when available_qty = 20	Application pre-check catches insufficient stock; error flash message "Insufficient stock!"; no allocation inserted	The application detected insufficient stock, displayed the "Insufficient stock!" message, and prevented the allocation.	Pass
TC-05	Equipment booking request submission	Valid equipment_id, date, time slot, purpose submitted by Student role	Booking record inserted with status='PENDING'; awaiting approval message shown	Booking was successfully created with PENDING status and the awaiting approval message was displayed.	Pass
TC-06	Faculty approval of a pending booking	Faculty clicks Approve for a PENDING booking	Booking status updated to 'APPROVED'; approved_by set to faculty user_id	The booking status changed to APPROVED and the faculty user's ID was recorded in approved_by.	Pass
TC-07	Faculty rejection of a pending booking	Faculty clicks Reject for a PENDING booking	Booking status updated to 'REJECTED'; rejected message shown	The booking status changed to REJECTED and the rejection message was displayed.	Pass
TC-08	Component return with Good condition	Submit return for allocation_id with returned_quantity=1, condition='Good'	RETURNS record inserted; EQUIPMENT_ALLOCATION.status set to 'Returned'; COMPONENTS.available_quantity incremented	The return record was successfully inserted, allocation status changed to Returned, and available component quantity increased.	Pass
TC-09	Component return with Burnt/Faulty condition	Submit return with condition='Burnt/Faulty'	RETURNS record inserted; EQUIPMENT_ALLOCATION.status updated; COMPONENTS.available_quantity NOT incremented (damaged beyond use)	The return was recorded and the allocation status was updated, while the available component quantity remained unchanged	Pass
TC-10	High-severity breakdown report	Submit breakdown for EQ001 with severity='High'	BREAKDOWNS record inserted; EQUIPMENT.status updated to 'Maintenance' automatically	The breakdown was recorded successfully and EQ001 status was automatically changed to Maintenance.	Pass
TC-11	Low-severity breakdown report	Submit breakdown for EQ001 with severity='Low'	BREAKDOWNS record inserted; EQUIPMENT.status remains unchanged (not set to Maintenance)	The breakdown was recorded and the equipment status remained unchanged.	Pass
TC-12	Maintenance job completion	Complete maintenance job with completion_date and action_taken	MAINTENANCE_JOBS.status = 'Completed'; EQUIPMENT.status = 'Available'; linked BREAKDOWNS.status = 'Resolved'	The maintenance job was marked Completed, the equipment status changed to Available, and the linked breakdown was marked Resolved.	Pass

Test ID	Feature	Input / Action	Expected Result	Actual Result	Status
TC-13	Low stock alert count in navigation bar	Load any page when available_quantity <= minimum_stock for at least one component	Low-stock badge visible in navbar with correct count	The low-stock badge was displayed in the navigation bar with the correct number of low-stock components.	Pass
TC-14	Equipment search by keyword	Search for "Robot" in equipment search bar	Equipment list filtered to show only items with "Robot" in equipment_name, equipment_id, or location	The search successfully filtered the equipment list and displayed matching records containing "Robot".	Pass
TC-15	User self-deletion prevention	Admin user attempts to delete their own active account	Warning flash message displayed; DELETE not executed	The system displayed a warning message and prevented the administrator from deleting their own account.	Pass
TC-16	Student role restriction on equipment add	Student user submits the equipment add form	Unauthorized flash message displayed; record not inserted	The system denied the operation, displayed an unauthorized message, and did not insert the equipment record.	Pass
TC-17	Duplicate email registration rejection	Attempt to register with email already existing in USERS table	Existing account check detects duplicate; warning flash message; redirect to login	The duplicate email was detected, a warning message was displayed, and the user was redirected to the login page.	Pass
TC-18	Reports – Low Stock Report execution	Navigate to /reports?report=low_stock	SQL executes correctly; table displays components with available_quantity <= minimum_stock	The low-stock report executed successfully and displayed components whose available quantity was at or below the minimum stock level.	Pass
TC-19	Reports – Equipment Utilization (GROUP BY)	Navigate to /reports?report=equipment_utilization	Report displays each equipment with COUNT of bookings and successful bookings	The equipment utilization report executed successfully and displayed booking counts and successful booking counts for each equipment item.	Pass
TC-20	Procurement status update triggers stock increase	Update requisition status to 'Received' for a requisition linked to component C001 with required_quantity=10	REQUISITIONS.status = 'Received'; COMPONENTS.total_quantity and available_quantity each incremented by 10	The requisition status changed to Received, and both total quantity and available quantity of C001 increased by 10.	Pass

Summary of Results. All 20 test cases passed successfully. The results demonstrate that the application correctly uses the designed database for equipment management, component allocation, bookings, returns, maintenance, reporting, notifications, and requisitions. The application also correctly enforces the defined business rules, including duplicate prevention, stock validation, booking approval and rejection, role-based authorization, equipment status updates, low-stock detection, and quantity updates during requisitions. Overall, the test results confirm that the implemented database operations and business rules function as intended.

CHAPTER 4

CONCLUSION

The ROBO-LAB project demonstrates the practical application of database management systems concepts to address a real and recurring operational problem in an academic robotics laboratory. The laboratory's dependency on informal, unstructured resource tracking methods was identified as the root cause of availability uncertainty, untracked borrowing, loss of maintenance history, and inefficient resource allocation across student projects. This report documents the design and implementation of a structured solution to these problems.

The project began with a conceptual Entity-Relationship model that captured nineteen distinct entities and their relationships, covering the complete lifecycle of laboratory resources from initial inventory registration through booking, approval, allocation, usage, return, breakdown, maintenance, and calibration. The ER model was then translated into a fully normalized logical relational schema. All nineteen relations were verified to satisfy Third Normal Form (3NF), ensuring that no redundant descriptive data is stored across tables and that all non-key attributes are determined solely by the primary key without transitive dependencies. The normalization process explicitly handled the separation of equipment categories, component categories, laboratories, vendors, and roles into dedicated lookup tables, and the use of foreign key joins to resolve descriptive attributes at query time.

The MySQL DDL implementation translates the logical design directly into a working database, with primary keys, foreign keys, UNIQUE constraints, NOT NULL constraints, CHECK constraints for quantity and cost validation, ENUM domains for status management, five SQL views for frequently executed analytical queries, and three triggers for automated stock management and equipment status transitions. The Python Flask web application

provides a complete interface to this database, implementing role-based access control, full CRUD operations across all modules, and a Reports Playground that executes eight pre-defined analytical queries demonstrating SELECT, multi-table JOIN, LEFT JOIN, GROUP BY, HAVING, UNION ALL, subquery, and aggregate function usage. The system's design directly answers the seven key operational questions identified in the problem statement: resource availability, current user, associated project, issue and return dates, maintenance status, vendor history, and low-stock conditions.

From a learning perspective, this project provided hands-on experience in ER modeling, relational schema design, database normalization theory, MySQL DDL and DML implementation, database trigger design, SQL view creation, complex query formulation, and the integration of a relational database with a web application through a Python backend. The ROBO-LAB system, while an academic prototype, demonstrates a practical and technically sound approach to laboratory resource management.

4.1 Limitations and Future Enhancements

Current Limitations:

- The system is a small-scale academic prototype and has not been tested or optimized for concurrent multi-user production environments.
- The notification mechanism is entirely database-driven and application-internal. No external email or SMS notification delivery is implemented.
- Inventory updates, specifically restocking and initial quantity entry, are performed manually by authorized staff. There is no automated integration with a purchasing or procurement system.
- The booking conflict validation (preventing overlapping bookings for the same equipment on the same date and time slot) is not yet implemented as an explicit database constraint or stored procedure; it relies on application-level logic.
- The authentication system does not implement password hashing in the current version; password storage is in plain text or a placeholder hash format. A production deployment would require proper bcrypt or PBKDF2 hashing.
- There is no mobile-native application; the interface is web-based and requires browser access.
- The fallback to SQLite means that MySQL-specific features (triggers, certain ENUM behaviors) may behave differently in the development environment.

Proposed Future Enhancements:

- **QR/Barcode-based Equipment Tracking:** Assign and print QR codes for each equipment asset and component storage bin to enable rapid scan-based check-in and check-out.
- **RFID Integration:** Deploy RFID readers at laboratory entry and exit points for passive, automated tracking of equipment movement.
- **IoT-based Equipment Monitoring:** Integrate microcontroller-based sensors to monitor equipment power state, usage hours, and environmental conditions in real time.
- **Email and SMS Notification Delivery:** Integrate SMTP and SMS gateway APIs to send automated alerts for booking approvals, overdue returns, and calibration due dates.
- **Mobile Application:** Develop a companion mobile application (Android/iOS) for student and technician access.
- **Automated Calibration Reminders:** Implement a scheduled background task (cron job) that automatically updates calibration statuses and generates notifications as due dates approach.
- **Advanced Analytics Dashboard:** Implement charts and trend visualizations for equipment utilization rates, breakdown frequency trends, and component consumption patterns across academic semesters.
- **Cloud Deployment:** Deploy the application on a cloud platform (such as AWS RDS for MySQL and EC2 for Flask) for institution-wide access.
- **Booking Conflict Prevention via Stored Procedure:** Implement a MySQL stored procedure or BEFORE INSERT trigger that explicitly checks for time-slot overlaps before permitting a new booking insertion.

REFERENCES

Sources are cited in the text using IEEE numbering and listed in order of first appearance.

- [1] C. J. Date, *An Introduction to Database Systems*, 8th ed. Boston, MA, USA: Addison-Wesley, 2004.
- [2] A. Silberschatz, H. F. Korth, and S. Sudarshan, *Database System Concepts*, 7th ed. New York, NY, USA: McGraw-Hill Education, 2020.
- [3] R. Ramakrishnan and J. Gehrke, *Database Management Systems*, 3rd ed. New York, NY, USA: McGraw-Hill, 2003.
- [4] Oracle Corporation, *MySQL 8.0 Reference Manual*. Accessed: Oct. 2026. [Online]. Available: <https://dev.mysql.com/doc/refman/8.0/en/>
- [5] Python Software Foundation, *Python 3 Documentation*. Accessed: Oct. 2026. [Online]. Available: <https://docs.python.org/3/>
- [6] Pallets Projects, *Flask Documentation*, ver. 3.x. Accessed: Oct. 2026. [Online]. Available: <https://flask.palletsprojects.com/>
- [7] Oracle Corporation, *MySQL Connector/Python Developer Guide*. Accessed: Oct. 2026. [Online]. Available: <https://dev.mysql.com/doc/connector-python/en/>

APPENDICES

APPENDIX A - COMPLETE MYSQL DDL/DML/QUERY SCRIPT

The complete MySQL script for the ROBO-LAB database is maintained in the project repository under `database/schema.sql` (DDL: CREATE TABLE, CREATE VIEW, CREATE TRIGGER statements), `database/seed.sql` (DML: INSERT statements for sample data), and `database/queries.sql` (query bank with 10 analytical queries for academic review).

File: `database/schema.sql` – Contains:

- CREATE DATABASE `robotics_lab`
- 19 CREATE TABLE statements with all constraints
- 5 CREATE OR REPLACE VIEW statements:
 - `vw_available_equipment`
 - `vw_active_allocations`
 - `vw_low_stock_components`
 - `vw_equipment_maintenance_history`
 - `vw_booking_summary`
- 3 CREATE TRIGGER statements:
 - `trg_after_allocation_insert` (deducts available quantity)
 - `trg_after_return_insert` (restores available quantity conditionally)
 - `trg_after_breakdown_insert` (sets equipment to Maintenance on High/Critical severity)

File: `database/seed.sql` – Contains INSERT statements for all 19 tables with realistic robotics laboratory sample data, including 8 equipment records, 12 component types, 8 users, 4 projects, 5 bookings, 8 allocations, 3 vendors, 2 breakdowns, 2 maintenance jobs, 3 calibrations, and 3 notifications.

File: `database/queries.sql` – Contains 10 analytical SQL queries demonstrating: 3-table and 4-table JOINS; LEFT JOIN with COALESCE; GROUP BY with COUNT and conditional SUM; HAVING clause; subquery with IN and NOT IN; DATEDIFF; UNION ALL for timeline aggregation.

The full script contents are available in the project source files and are not reproduced in full here to avoid redundancy. For the viva review, the files should be executed in the sequence: schema.sql → seed.sql.

APPENDIX B - PYTHON FLASK SOURCE CODE REFERENCE

The Python Flask application source code is organized as follows:

```
SQL_PBL/
├── app.py                # Main Flask application (~1,396 lines, ~40 routes)
├── config.py             # Database connection configuration (MySQL + SQLite fallback)
├── db.py                 # Database helper functions: query_db(), execute_db()
├── requirements.txt      # Python dependencies (Flask 3.0.3, mysql-connector-python
8.4.0)
├── .env.example          # Environment variable template for DB credentials
├── database/
│   ├── schema.sql        # Complete DDL (tables, views, triggers)
│   ├── seed.sql          # Sample data DML
│   ├── queries.sql       # Analytical query bank
│   └── robolab.db        # SQLite fallback database
├── templates/           # Jinja2 HTML templates (17 screens)
│   ├── layout.html       # Base template with navigation
│   ├── login.html        # Authentication screen
│   ├── dashboard.html    # Role-tailored dashboard
│   ├── equipment.html    # Equipment catalog
│   ├── components.html   # Component inventory
│   ├── bookings.html     # Booking management
│   ├── allocations.html  # Component issuance and returns
│   ├── projects.html     # Project management
│   ├── maintenance.html  # Maintenance and breakdowns
│   ├── calibrations.html # Calibration schedules
│   ├── vendors.html      # Vendor management
│   ├── reports.html      # Reports playground
│   ├── procurement.html  # Procurement/requisitions
│   ├── users.html        # User management
│   └── profile.html      # User profile
├── static/
│   └── js/main.js        # Frontend JavaScript
│       [css files]
```

Repository: https://github.com/MSN-2007/Lab_Management-.git

To Run Locally:

```
# 1. Install dependencies
pip install -r requirements.txt

# 2. Configure environment (copy and edit)
cp .env.example .env

# 3. Initialize MySQL database (if MySQL available)
mysql -u root -p < database/schema.sql
mysql -u root -p robotics_lab < database/seed.sql

# 4. Run Flask application
python app.py
# Application available at: http://127.0.0.1:5000
```

APPENDIX C - ADDITIONAL TEST DATA, SCREENSHOTS AND OUTPUTS

C.1 Sample SQL Query Outputs

The following query outputs are representative results from the implemented database populated with seed data from `database/seed.sql`. Actual outputs may vary depending on the state of the database at the time of execution.

Query: Equipment Availability Report

```
SELECT e.equipment_id, e.equipment_name, ec.category_name, l.lab_name, e.status
FROM EQUIPMENT e
JOIN EQUIPMENT_CATEGORIES ec ON e.category_id = ec.category_id
JOIN LABORATORIES l ON e.laboratory_id = l.lab_id
WHERE e.status = 'Available'
ORDER BY e.equipment_name ASC;
```

Expected output (based on seed data):

equipment_id	equipment_name	category_name	lab_name	status
EQ005	Bantam PCB Milling Machine	Subtractive Fabrication	Rapid Prototyping & Fabrication Lab	Available
EQ006	DJI Matrice Autonomous Drone Base	Autonomous Mobile Robots	Advanced Mechatronics & Drones Lab	Available
EQ001	6-DOF Articulated Robotic Arm	Robotic Arms & Manipulators	Central Robotics & Automation Lab	Available
EQ003	Rigol 100MHz Digital Oscilloscope	Test & Measurement	Central Robotics & Automation Lab	Available
EQ007	Delta High-Speed Pick & Place Robot	Robotic Arms & Manipulators	Central Robotics & Automation Lab	Available

Table 19. Output of the Equipment Availability Report

equipment_id	equipment_name	category_name	lab_name	status
EQ001	6-DOF Articulated Robotic Arm	Robotic Arms & Manipulators	Central Robotics & Automation Lab	Available
EQ005	Bantam PCB Milling Machine	Subtractive Fabrication	Rapid Prototyping & Fabrication Lab	Available
EQ007	Delta High-Speed Pick & Place Robot	Robotic Arms & Manipulators	Central Robotics & Automation Lab	Available
EQ006	DJI Matrice Autonomous Drone Base	Autonomous Mobile Robots	Advanced Mechatronics & Drones Lab	Available
EQ003	Rigol 100MHz Digital Oscilloscope	Test & Measurement	Central Robotics & Automation Lab	Available

Query: Low Stock Components

```
SELECT c.component_id, c.component_name, c.available_quantity, c.minimum_stock,
       (c.minimum_stock - c.available_quantity) AS shortage_units
FROM COMPONENTS c
WHERE c.available_quantity <= c.minimum_stock
ORDER BY shortage_units DESC;
```

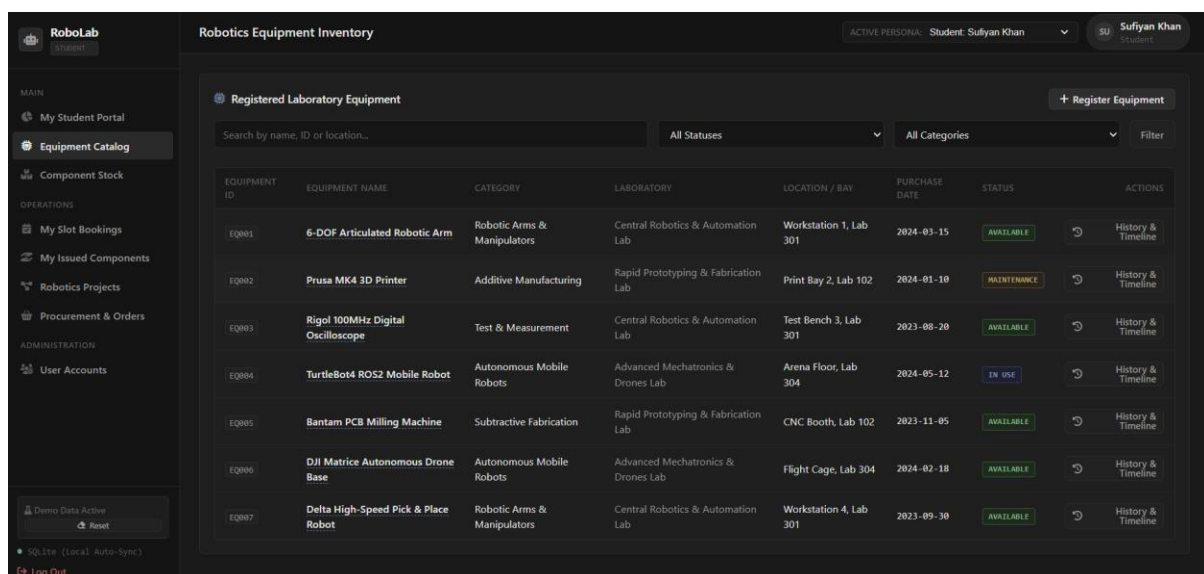
Expected output (based on seed data):

component_id	component_name	available_qty	minimum_stock	shortage_units
C010	OAK-D Spatial AI Camera Module	1	2	1
C005	MPU-6050 6-DOF IMU	8	8	0
C006	RPLiDAR A1M8 360 Degree Laser Scanner	2	2	0

Table 20. Output of the Low Stock Components

component_id	component_name	available_qty	minimum_stock	shortage_units
C010	OAK-D Spatial AI Camera Module	1	2	1
C005	MPU-6050 6-DOF IMU	8	8	0
C006	RPLiDAR A1M8 360 Degree Laser Scanner	2	2	0

C.2 Application Screenshots



RoboLab

Admin

MAIN

Dashboard

Equipment Catalog

Component Stock

OPERATIONS

Equipment Bookings

Project Allocations

Robotics Projects

Procurement & Orders

ADMINISTRATION

User Accounts

Maintenance

Calibrations

Vendors

Demo Data Active

Reset

SQLite (Local Auto-Sync)

Log Out

Robotics Equipment Inventory

ACTIVE PERSONA: Admin: Dr. Rajesh Sharma

DR: Dr. Rajesh Sharma

Registered Laboratory Equipment

+ Register Equipment

Search by name, ID or location...

All Statuses

All Categories

Filter

EQUIPMENT ID	EQUIPMENT NAME	CATEGORY	LABORATORY	LOCATION / BAY	PURCHASE DATE	STATUS	ACTIONS
EQ001	6-DOF Articulated Robotic Arm	Robotic Arms & Manipulators	Central Robotics & Automation Lab	Workstation 1, Lab 301	2024-03-15	AVAILABLE	🔄 History & Timeline
EQ002	Prusa MK4 3D Printer	Additive Manufacturing	Rapid Prototyping & Fabrication Lab	Print Bay 2, Lab 102	2024-01-10	MAINTENANCE	🔄 History & Timeline
EQ003	Rigol 100MHz Digital Oscilloscope	Test & Measurement	Central Robotics & Automation Lab	Test Bench 3, Lab 301	2023-08-20	AVAILABLE	🔄 History & Timeline
EQ004	TurtleBot4 ROS2 Mobile Robot	Autonomous Mobile Robots	Advanced Mechatronics & Drones Lab	Arena Floor, Lab 304	2024-05-12	IN USE	🔄 History & Timeline
EQ005	Bantam PCB Milling Machine	Subtractive Fabrication	Rapid Prototyping & Fabrication Lab	CNC Booth, Lab 102	2023-11-05	AVAILABLE	🔄 History & Timeline
EQ006	DJI Matrice Autonomous Drone Base	Autonomous Mobile Robots	Advanced Mechatronics & Drones Lab	Flight Cage, Lab 304	2024-02-18	AVAILABLE	🔄 History & Timeline
EQ007	Delta High-Speed Pick & Place Robot	Robotic Arms & Manipulators	Central Robotics & Automation Lab	Workstation 4, Lab 301	2023-09-30	AVAILABLE	🔄 History & Timeline

Equipment Booking & Slot Management

ACTIVE PERSONA: Admin: Dr. Rajesh Sharma

DR: Dr. Rajesh Sharma

Equipment Slot Reservations

+ New Booking Request

All Bookings

Pending Approvals

Approved Slots

In Session (Active)

Completed & Returned

BOOKING ID	EQUIPMENT	STUDENT / RESEARCHER	PROJECT	SLOT DATE & TIME	PURPOSE / EXPERIMENT	STATUS	WORKFLOW ACTION
#3	Rigol 100MHz Digital Oscilloscope Test Bench 3, Lab 301	Pooja Nair RA21110030102	Autonomous Line & Maze Following Robot	2026-09-02 11:00:00 - 13:00:00	PWM frequency and motor back-EMF analysis on oscilloscope	PENDING	Approve Reject
#2	TurtleBot4 ROS2 Mobile Robot Arena Floor, Lab 304	Rahul Sundaram RA21110030103	Autonomous Mobile Delivery Robot (ROS2)	2026-09-01 14:00:00 - 17:00:00	SLAM map generation in lab corridor using LIDAR	IN USE	✓ Mark Returned
#1	6-DOF Articulated Robotic Arm Workstation 1, Lab 301	Sufiyan Khan RA21110030101	Vision Guided 6-DOF Pick & Place Robotic Arm	2026-09-01 10:00:00 - 12:00:00	Inverse kinematics calibration and gripper testing with vision pipeline	APPROVED	▶ Start Session
#5	Bantam PCB Milling Machine CNC Booth, Lab 102	Pooja Nair RA21110030102	Autonomous Line & Maze Following Robot	2026-08-28 14:00:00 - 16:00:00	Milling motor driver custom shield PCB	RETURNED	—
#4	6-DOF Articulated Robotic Arm Workstation 1, Lab 301	Rahul Sundaram RA21110030103	Vision Guided 6-DOF Pick & Place Robotic Arm	2026-08-25 09:00:00 - 11:30:00	Repeatability and precision test for pick and place	RETURNED	—

Equipment Booking & Slot Management

ACTIVE PERSONA: Student

Equipment Slot Reservations

All Bookings Pending Approvals

BOOKING ID EQUIPMENT PURPOSE / EXPERIMENT DESCRIPTION

#1 6-DOF Articulated Robotic Arm Workstation 1, Lab 301 Inverse kinematics gripper testing with

Book Equipment Slot

EQUIPMENT TO RESERVE *

6-DOF Articulated Robotic Arm (EQ001 - Available)

STUDENT / RESEARCHER *

Pooja Nair (RA21110030102)

ASSOCIATED PROJECT

-- None (Independent Lab Activity) --

DATE OF USAGE *

mm/dd/yyyy

START TIME *

--:-- --

END TIME *

--:-- --

PURPOSE / EXPERIMENT DESCRIPTION *

Describe the experiments, algorithms, or testing plan...

Cancel Submit Booking Request

RoboLab

STUDENT

Academic Robotics Projects & Teams

ACTIVE PERSONA: Student: Sufiyan Khan

Sufiyan Khan Student

MAIN

My Student Portal

Equipment Catalog

Component Stock

OPERATIONS

My Slot Bookings

My Issued Components

Robotics Projects

Procurement & Orders

ADMINISTRATION

User Accounts

Active Student Robotics Projects

Register New Project

Project #1

ACTIVE

Autonomous Line & Maze Following Robot

High-speed autonomous PID line follower with obstacle evasion and intersection maze solving.

Guide Faculty: Dr. Ananya Iyer

Timeline: 2026-08-01 → 2026-11-30

TEAM MEMBERS (2)

Sufiyan Khan (RA21110030101) Project Lead / Firmware

Pooja Nair (RA21110030102) Hardware & Circuit Designer

4 Active Components

View Allocations

Project #2

ACTIVE

Vision Guided 6-DOF Pick & Place Robotic Arm

Industrial object sorting system utilizing OpenCV and YOLO object detection for pick-and-place manipulation.

Guide Faculty: Prof. Arvind Menon

Timeline: 2026-07-15 → 2026-12-15

TEAM MEMBERS (2)

Sufiyan Khan (RA21110030101) Kinematics & Control

Rahul Sundaram (RA21110030103) Computer Vision Specialist

0 Active Components

View Allocations

Project #3

ACTIVE

Autonomous Mobile Delivery Robot (ROS2)

Indoor autonomous navigation robot using 2D LIDAR SLAM and Nav2 stack on TurtleBot.

Guide Faculty: Prof. Arvind Menon

Timeline: 2026-08-10 → 2026-12-20

TEAM MEMBERS (2)

Rahul Sundaram (RA21110030103) SLAM & Navigation Lead

Sneha Patel (RA21110030104) Perception & ROS2 Engineer

2 Active Components

View Allocations

Project #4

ACTIVE

Quadrotor Drone Indoor Obstacle Avoidance

Autonomous flight stabilization using optical flow sensors and ultrasonic altitude lock.

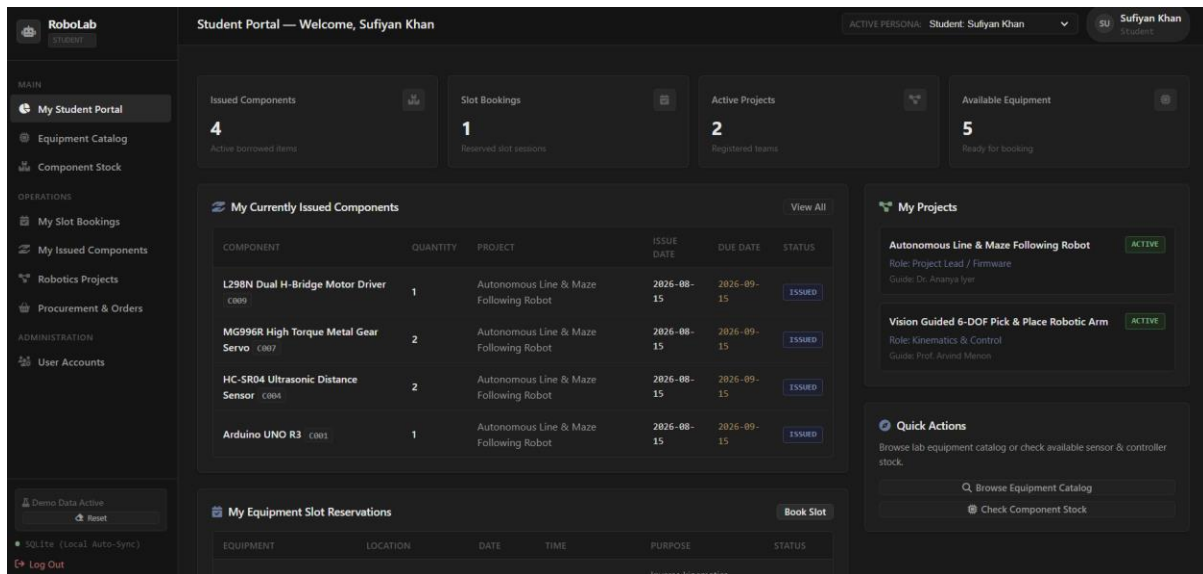
Guide Faculty: Dr. Ananya Iyer

Demo Data Active

Reset

SQLite (Local) Auto-Sync

Log Out



C.3 Database Trigger Verification

To verify that `trg_after_allocation_insert` correctly decrements `available_quantity`:

```
-- Before allocation: Check C001 available_quantity
SELECT component_id, available_quantity FROM COMPONENTS WHERE component_id = 'C001';

-- Issue allocation of 3 units to student 5, project 1
INSERT INTO EQUIPMENT_ALLOCATION
    (component_id, user_id, project_id, quantity, issue_date, expected_return_date,
    status)
VALUES ('C001', 5, 1, 3, CURDATE(), DATE_ADD(CURDATE(), INTERVAL 30 DAY), 'Issued');

-- After allocation: Verify decrement
SELECT component_id, available_quantity FROM COMPONENTS WHERE component_id = 'C001';
-- Expected: available_quantity reduced by 3
```